

1 実験の進め方

本実験では、課題を **練習課題** と **必須課題** に分けて進める。練習課題は、実験環境や操作方法に慣れ、結果の見方やソースコードの流れを理解することを目的とした課題である。まずは練習課題に取り組み、実験の基本的な手順を確認することが重要である。

これに対し、必須課題は、実験内容の理解を確認するために取り組む課題であり、レポート提出の対象となる課題である。したがって、必須課題については、単に実行するだけでなく、得られた結果を整理し、必要な考察を加えたうえでまとめることが求められる。

本資料に掲載する一部のソースコードには、実行環境や対象チェーンに応じて変更すべき箇所を*___*として示している。この*___*の部分を各自の実験条件に合わせて置き換えたうえで、ソースコードを実行すること。

練習課題

操作方法や結果の見方に慣れるための課題である。まずはこちらを実施して、ソースコードの流れを確認すること。

必須課題

レポート提出の対象となる課題である。

2 実験全体の環境構築

本実験では、WSL 上の Kali Linux 環境を用いて作業を行う。まず、Windows の管理者権限付きコマンドプロンプトで WSL を利用し、Kali Linux を導入する。導入コマンドをコマンド 1 に示す。

コマンド 1 WSL を利用した Kali Linux の導入コマンド

```
1 wsl --install -d kali-linux
```

VS Code から wsl と Remote SSH をインストールする。

導入後は、Kali Linux 上で必要なパッケージを更新し、Python 仮想環境および Java 実行環境などを整備する。Python 関連パッケージの導入コマンドをコマンド 2 に示す。

コマンド 2 Python 環境整備コマンド

```
1 sudo apt update
2 sudo apt upgrade -y
3 sudo apt install -y python3-pip python3-venv openjdk-11-jdk unzip npm jq
```

作業ディレクトリと Python 仮想環境の作成コマンドをコマンド 3 に示す。

```
1 mkdir ~/temp
```

```
1 cd ~/temp
2 git clone https://github.com/dsg-titech/simblock
```

```

3 export JAVA_HOME=/usr/lib/jvm/java-11-openjdk-amd64
4 export PATH=$JAVA_HOME/bin:$PATH
5 cd ~/simblock
6 chmod +x gradlew
7 ./gradlew clean
8 ./gradlew build
9 ./gradlew :simulator:run

```

コマンド 3 作業ディレクトリと Python 仮想環境の作成コマンド

```

1 cd ~/temp
2 git clone https://github.com/yanagihalab/sus_BC2
3 cd sus_BC2
4 python3 -m venv venv
5 source venv/bin/activate
6 pip install -r requirements.txt

```

2.1 Kali Linux 上での injectived 実行環境の整備

本実験を Kali Linux 上で実施するためには、Injective の CLI 兼ノードデーモンである injectived を利用可能な状態にする必要がある。本環境では、Linux x86_64 向けの事前ビルド済みバイナリを用いて導入する。

2.1.1 injectived の導入

injectived の取得と展開コマンドをコマンド 4 に示す。

コマンド 4 injectived の取得と展開コマンド

```

1 cd ~/temp
2 wget https://github.com/InjectiveFoundation/injective-core/releases/latest/download/
↪ linux-amd64.zip
3 unzip linux-amd64.zip
4 chmod +x injectived
5 sudo mv injectived /usr/local/bin/

```

injectived の実行時に共有ライブラリが見つからない場合は、同梱されている libwasmvm.x86_64.so をシステムのライブラリパスへ配置する。共有ライブラリの配置と動作確認コマンドをコマンド 5 に示す。

コマンド 5 共有ライブラリの配置と動作確認コマンド

```

1 sudo mv libwasmvm.x86_64.so /usr/lib/
2 sudo ldconfig
3 injectived version

```

動作確認結果の例を出力 6 に示す。

出力 6 injectived の動作確認結果

```

Version v1.18.3 (b18483b)
Compiled at 20260331-2102 using Go go1.23.9 (amd64)

```

2.2 wallet の作成

まず、鍵名 `testwallet` のウォレットを作成する。ウォレット作成コマンドをコマンド 26 に示す。

コマンド 7 ウォレット作成コマンド

```
1 injectived keys add testwallet --keyring-backend test
```

すでにウォレットを作成済みの場合は、鍵を確認し、使用するアドレスを取得する。鍵情報の確認コマンドをコマンド 8 に示す。

コマンド 8 鍵情報の確認コマンド

```
1 injectived keys list --keyring-backend test
2 injectived keys show testwallet -a --keyring-backend test
```

次に、Injective testnet（テストネット）用の共通設定を環境変数として与える。環境変数の設定コマンドをコマンド 27 に示す。

コマンド 9 Injective testnet 用環境変数の設定コマンド

```
1 export CHAIN_ID=injective-888
2 export NODE=https://injective-testnet-rpc.publicnode.com:443
3 export GAS_PRICES=500000000inj
4 export GAS=1000000
5 export KEY_NAME=mykey
6 export MY_ADDR=$(injectived keys show $KEY_NAME -a --keyring-backend test)
7 export SUBDENOM=mytoken
```

`faucet`（フォーセット）実行後に、残高が反映されたかを確認するため、ウォレット残高を確認する。残高確認コマンドをコマンド 28 に示す。

コマンド 10 faucet 実行後の残高確認コマンド

```
1 injectived query bank balances $MY_ADDR --node=$NODE
```

出力結果に `inj` の残高が含まれていれば、`faucet` による入金 that 反映されていると判断できる。

2.2.1 Docker Engine の導入

本実験では、CW20 コントラクトのビルドに `cosmwasm/workspace-optimizer` を用いる場合がある。このビルド方法では Docker を利用するため、事前に Docker Engine を利用可能な状態にしておく必要がある。Docker Engine の導入コマンドをコマンド 11 に示す。

コマンド 11 Docker Engine の導入コマンド

```
1 sudo apt update
2 sudo apt install -y docker.io
3 sudo systemctl enable docker
4 sudo systemctl start docker
5 sudo docker --version
```

一般ユーザで Docker を実行する場合は、必要に応じて現在のユーザを `docker` グループに追加する。Docker グループへの追加コマンドをコマンド 12 に示す。

コマンド 12 Docker グループへのユーザ追加コマンド

```
1 sudo usermod -aG docker $USER
2 newgrp docker
3 docker run hello-world
```

2.3 CW20 コントラクト用の Docker / CosmWasm 環境構築

CW20 コントラクトをビルドするためには、CosmWasm コントラクトを WASM 形式へ変換するビルド環境が必要である。本実験では、ホスト環境に Rust / Cargo の詳細なビルド環境を直接構築するのではなく、`cosmwasm/workspace-optimizer` Docker イメージを用いて最適化済み WASM を生成する。

また、本実験では、CosmWasm の `cw-plus` に含まれる `cw20-base` をサンプルコードとして利用する。`cw20-base` は、CW20 トークンの基本実装であり、トークン名、シンボル、初期残高、mint 権限などを設定して利用できる。

2.3.1 Docker Engine の導入

`cosmwasm/workspace-optimizer` を利用するためには、Docker Engine が必要である。Docker Engine の導入コマンドをコマンド 13 に示す。

コマンド 13 Docker Engine の導入コマンド

```
1 sudo apt update
2 sudo apt install -y docker.io
3 sudo systemctl enable docker
4 sudo systemctl start docker
5 sudo docker --version
```

一般ユーザで Docker を実行する場合は、必要に応じて現在のユーザを `docker` グループに追加する。Docker グループへの追加コマンドをコマンド 14 に示す。

コマンド 14 Docker グループへのユーザ追加コマンド

```
1 sudo usermod -aG docker $USER
2 newgrp docker
3 docker run hello-world
```

2.3.2 CW20 サンプルコードの取得

本実験では、CW20 コントラクトを一から実装するのではなく、CosmWasm が提供する `cw-plus` リポジトリ内の `cw20-base` をサンプルコードとして利用する。`cw-plus` の取得コマンドをコマンド 15 に示す。

コマンド 15 CW20 サンプルコードの取得コマンド

```
1 cd ~/temp
2 git clone https://github.com/CosmWasm/cw-plus.git
3 cd cw-plus
4 ls
```

`cw20-base` コントラクトは、`cw-plus/contracts/cw20-base` に含まれている。確認コマンドをコマンド 16 に示す。

コマンド 16 cw20-base ディレクトリの確認コマンド

```
1 ls contracts/cw20-base
```

2.3.3 workspace-optimizer によるビルド

取得した cw20-base コントラクトを WASM 形式へビルドする。本実験では, cosmwasm/workspace-optimizer を用いて Docker 上で最適化ビルドを行う。

cosmwasm/workspace-optimizer は, CosmWasm コントラクトを再現性のある環境でビルドし, WASM サイズの最適化を行うための Docker イメージである。特に, cw-plus のように複数のコントラクトを含む workspace 形式のリポジトリをビルドする場合に適している [?].

workspace-optimizer を用いる場合は, contracts/cw20-base ディレクトリではなく, cw-plus のルートディレクトリで実行する点に注意する。ビルドコマンドをコマンド 17 に示す。

コマンド 17 workspace-optimizer による CW20 ビルドコマンド

```
1 cd ~/temp/cw-plus
2
3 docker run --rm \
4   -v "$(pwd)":/code \
5   --mount type=volume,source="$(basename "$(pwd)")_cache",target=/code/target \
6   --mount type=volume,source=registry_cache,target=/usr/local/cargo/registry \
7   cosmwasm/workspace-optimizer:0.17.0
```

ビルドが成功すると, artifacts ディレクトリに最適化済みの WASM ファイルが出力される。cw20-base の場合は, 通常, artifacts/cw20_base.wasm が生成される。生成物の確認コマンドをコマンド 18 に示す。

コマンド 18 CW20 WASM 生成物の確認コマンド

```
1 ls -lh artifacts/
2 ls -lh artifacts/cw20_base.wasm
```

2.3.4 コントラクトコードの保存

次に, ビルド済みの WASM ファイルを Injective チェーンへ保存する。workspace-optimizer によって生成された artifacts/cw20_base.wasm を指定する。保存コマンド例をコマンド 19 に示す。

コマンド 19 最適化済み CW20 コントラクトの保存コマンド例

```
1 injectived tx wasm store artifacts/cw20_base.wasm \
2   --from=$KEY_NAME \
3   --chain-id=$CHAIN_ID \
4   --node=$NODE \
5   --gas-prices=$GAS_PRICES \
6   --gas=3000000
```

保存に成功すると, トランザクション結果から code ID を確認できる。以降のインスタンス化では, この code ID を指定する。

2.4 SimBlock の動作環境構築

SimBlock を利用するためには, Java 実行環境が必要である。本実験では, SimBlock に付属する Gradle Wrapper を用いてビルドおよび実行を行う。セットアップコマンドをコマンド 20 に示す。

```

1 sudo apt update
2 sudo apt install -y openjdk-11-jdk unzip git
3 export JAVA_HOME=/usr/lib/jvm/java-11-openjdk-amd64
4 export PATH=$JAVA_HOME/bin:$PATH
5 cd ~/simblock
6 chmod +x gradlew
7 ./gradlew clean
8 ./gradlew build
9 ./gradlew :simulator:run

```

3 Day1

3.1 Cosmos 系ブロックチェーンの分析

Cosmos は、複数のブロックチェーンが相互に連携することを目的として設計されたブロックチェーンエコシステムである [1]。従来のブロックチェーンは、それぞれが独立して動作することが多く、異なるチェーン間で資産やデータをやり取りすることは容易ではなかった。これに対し Cosmos では、ブロックチェーン同士の相互運用性を重視し、独立した複数のチェーンを接続するための仕組みが整備されている [1, 3]。

Cosmos の基盤技術の一つが Cosmos SDK である [2]。Cosmos SDK は、ブロックチェーンを構築するためのモジュール群を提供する開発基盤であり、送金、アカウント管理、ステーキング、ガバナンスなどの基本機能を組み合わせることで、独自のブロックチェーンを比較的容易に構築できる [2]。このため、開発者は一からすべてを実装するのではなく、必要な機能を選択・拡張しながら独自チェーンを設計できる [2]。

また、Cosmos では Tendermint BFT を起源とするコンセンサス技術が採用されてきた [1]。この系統のコンセンサス機構では、ブロック生成と合意形成が効率的に行われ、比較的短い時間でブロックを確定できるという特徴がある [1]。そのため、トランザクションの確定時間やファイナリティの観点からも、実験対象として扱いやすいブロックチェーン基盤である。

さらに、Cosmos エコシステムにおいて重要な役割を果たすのが IBC (Inter-Blockchain Communication) である [3, 4]。IBC は、異なるブロックチェーン間でトークンやメッセージを安全にやり取りするための仕組みであり、Cosmos の相互運用性を支える中核技術である [3, 4]。この仕組みによって、各チェーンは独立性を保ちながらも、必要に応じて他のチェーンと連携できる [3]。

Cosmos では、このような相互接続された複数のブロックチェーン群を、しばしば「ブロックチェーンのインターネット」として捉える [1]。これは、インターネットにおいて異なる計算機ネットワーク同士が相互接続され、標準化された通信手順によって情報をやり取りする構造に対応する考え方に基づくものである。すなわち、Cosmos は単一の巨大なチェーンにすべての機能を集約するのではなく、目的ごとに設計された独立チェーンを相互接続し、全体として一つの大きなネットワークを構成することを目指している [1, 3]。IBC は、この「ブロックチェーンのインターネット」を実現するための通信基盤として位置づけられる [3, 4]。

図 1 および図 2 に、インターネットとブロックチェーンのインターネットの概念比較を示す。インターネットが複数のネットワークを相互接続することで全体として一つの通信基盤を構成しているのと同様に、Cosmos では複数の独立チェーンを接続することで、全体として一つの相互運用ネットワークを構成することを目指している [1, 3]。

本実験では、このような Cosmos の特徴を踏まえ、ブロック、トランザクション、アドレス、残高、手数料

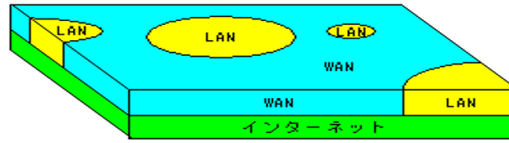


図1 インターネット, WAN, LAN の概念的なイメージ

出典：Wikipedia「Wide Area Network」[15]

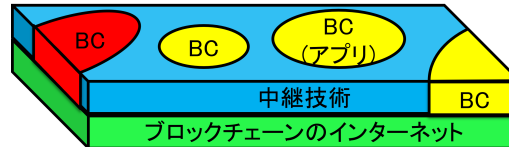


図2 ブロックチェーンのインターネットの概念的なイメージ

などの基本情報を確認しながら、Cosmos 系ブロックチェーンの構造と動作を理解することを目的とする。単に操作手順をなぞるのではなく、各情報がどのような意味を持ち、ブロックチェーン全体の中でどのような役割を果たしているかを意識しながら分析することが重要である。

3.2 Cosmos 系チェーンのブロックデータ取得と分析

本実験では、まず Cosmos 系チェーンのブロックデータを取得し、その後、保存済みの JSON 群に対して分析スクリプトを実行する。この作業の目的は、ブロックチェーン上に記録されている基本情報を把握し、後続のトークン実装や実験結果の解釈に必要な前提知識を得ることである。

Cosmos 系チェーンでは、各ブロックにブロック高、生成時刻、提案者アドレス、トランザクション群、署名情報などが含まれる [5]。また、各高さにおけるバリデータ集合（Validator Set）を取得することで、その時点でどのバリデータ（Validator）がネットワークの合意形成に参加していたかを確認できる [6, 5]。したがって、ブロックデータとバリデータデータをあわせて取得することで、単なるブロック列だけではなく、その背後にある合意形成の状態も分析対象とすることができる。

3.2.1 取得対象のデータ

本実験で取得するデータは、大きく分けて次の 2 種類である。

- ブロック情報
- バリデータ情報

ブロック情報には、各高さにおけるブロックヘッダ、ブロック本文、トランザクション、署名情報などが含まれる [5]。一方、バリデータ情報には、各バリデータのアドレス、投票力（Voting Power）、提案者優先度（Proposer Priority）などが含まれる [6, 5]。これらを同じ高さに対して対応付けて保存することで、後続の分析でブロック生成者とバリデータ状態の関係を調べる事が可能となる。

3.2.2 データ取得の流れ

最初に、RPC ノード (Remote Procedure Call node) のエンドポイントに問い合わせる最新ブロック高を取得する。次に、その最新高さを起点として、指定したブロック数だけ過去にさかのぼりながら、各高さについてブロック情報を取得する。さらに、同じ高さ (ブロック番号) を指定してバリデータ情報を取得し、1 つの JSON ファイルにまとめて保存する。

このとき、各ファイルは 1 ブロックに対応する形で保存されるため、後から特定のブロック高だけを個別に確認したり、複数のブロックをまとめて機械的に分析したりしやすい構造となる。また、取得対象を最新から過去方向へ一定数に限定することで、大規模なブロックチェーン全体を一度に扱うのではなく、比較的新しい区間に注目した分析を行える。

3.2.3 保存形式

取得したデータは、各ブロック高ごとに JSON ファイルとして保存する。ファイル名にはブロック高を含めることで、どのファイルがどの高さに対応するかを容易に識別できるようにする。各 JSON ファイルには、少なくとも次の 2 つの要素が含まれる。

- block_info
- validators

この構造により、1 つのファイルを開くだけで、その高さにおけるブロック内容とバリデータ集合を同時に確認できる。また、後続の分析スクリプトにおいても、統一された形式でデータを読み込むことができる。

3.2.4 分析スクリプト実行の目的

取得した JSON 群に対して分析スクリプトを実行する目的は、各ブロックに含まれる情報を定量的に整理し、傾向や規則性を把握することである。ブロックデータをそのまま読むだけでは、大量の JSON から全体傾向を把握することは容易ではない。そこで、分析スクリプトを用いて必要な情報を抽出し、CSV 形式へまとめることで、後から表計算ソフトや可視化ツールを用いて二次分析しやすくする。

分析スクリプトでは、各 JSON ファイルを読み込み、たとえば次のような項目を取り出して整理する。

- ブロック高
- タイムスタンプ
- チェーン ID
- 提案者アドレス (Proposer Address)
- トランザクション数
- 署名数
- バリデータ数
- Voting Power の最大値・最小値・合計
- Proposer Priority の最大値・最小値

これらの項目をブロックごとに表形式へまとめることで、チェーンの動作を定量的に観察できるようになる。特に、Proposer Address と Proposer Priority の関係を見ることで、各ブロックの提案者 (Proposer) がどのような優先度のもとで選ばれているかを確認できる。

この分析スクリプトの特徴の一つは、各ブロック単体だけでなく、直前のブロックとの関係も調べている点である。具体的には、あるブロックの Proposer Address が、前ブロックにおいて最も高い Proposer Priority を持っていたバリデータと一致するかを確認している。さらに、一致しない場合には、その Proposer が前ブロック時点で何位の Proposer Priority を持っていたかを記録している。

このような分析により、単純に「提案者が誰であったか」を記録するだけでなく、提案者選出の傾向や Priority の推移に関する観察が可能となる。その結果、ブロック提案者の選出規則や、実際のチェーン運用における偏りの有無を考察しやすくなる。

分析結果は最終的に CSV ファイルとして保存する。CSV には、ブロックごとの主要な指標が 1 行ずつ記録されるため、後からソート、集計、可視化を行うことが容易である。また、分析スクリプトでは、大きなネスト構造を持つ元の validators 配列などは削除したうえで保存するため、CSV として扱いやすい列構造に整理される。

以上のように、本作業は単にブロックデータを収集するだけでなく、それを分析可能な形式へ変換し、Cosmos 系チェーンの構造と動作を理解するための基盤を整えるものである。これにより、後続の CW20 実装や Injective 上での実験に進む前に、ブロックチェーン上のデータがどのように構成され、どのような観点で解析できるかを具体的に把握できる。したがって、本節は後続実験の準備段階であると同時に、Cosmos 系ブロックチェーンのデータ理解そのものを目的とする重要な工程である。

3.3 ブロック取得スクリプト

Cosmos 系チェーンのブロック情報およびバリデータ情報を取得するためのスクリプトをソースコード 21 に示す。このスクリプトでは、まず RPC から最新ブロック高を取得し、その後、指定したブロック数だけ過去にさかのぼりながら、各高さについてブロック情報とバリデータ情報を取得する。RPC のエンドポイント (BASE_URL) は以下から共同実験者と重複しないよう選択し、実行すること。

- **Osmosis:** <https://osmosis-rpc.publicnode.com:443> [必須選択]
- **Cosmos Hub:** <https://cosmos-rpc.publicnode.com:443>
- **Terra:** <https://terra-rpc.publicnode.com:443>
- **Juno:** <https://juno-rpc.publicnode.com:443>
- **Cronos:** <https://cronos-rpc.publicnode.com:443>
- **Evmos:** <https://evmos-rpc.publicnode.com:443>
- **Injective:** <https://injective-rpc.publicnode.com:443>

取得したデータは、高さごとに JSON ファイルとして保存する。これにより、後続の分析スクリプトが統一形式でデータを読み込めるようになる。

ソースコード 21 ブロック取得スクリプト (1-1)

```
1 import os
2 import requests
3 import json
4 import time
5 import re
6 from tqdm import tqdm
7 from urllib.parse import urlparse
8
```

```

9  BASE_URL = "*_**"
10
11  BASE_URL_BLOCK = *_** + "/block"
12  BASE_URL_BLOCK_RESULTS = *_** + "/block_results"
13  BASE_URL_VALIDATORS = *_** + "/validators"
14
15  PER_PAGE = 100
16  TOTAL_PAGES = 1
17  RETRY_LIMIT = 100
18  SLEEP_TIME = 1
19  BLOCK_COUNT = *_**
20
21  parsed_base = urlparse(BASE_URL)
22  hostname = parsed_base.hostname or ""
23
24  m = re.match(r"^([a-zA-Z0-9\-\-])+-rpc", hostname)
25  BASE_NAME = m.group(1) if m else hostname.split(".")[0]
26
27  SAVE_DIR = *_**
28
29  os.makedirs(SAVE_DIR, exist_ok=True)
30
31  headers = {"User-Agent": "Mozilla/5.0"}
32
33
34  def get_with_retry(url, timeout=10, retry_limit=RETRY_LIMIT, sleep_time=SLEEP_TIME):
35      retries = 0
36      while retries < retry_limit:
37          try:
38              response = requests.get(url, headers=headers, timeout=timeout)
39              response.raise_for_status()
40              return response.json()
41          except requests.exceptions.RequestException as e:
42              retries += 1
43              if retries >= retry_limit:
44                  raise e
45              time.sleep(sleep_time)
46
47
48  def get_latest_height():
49      latest_block = get_with_retry(BASE_URL_BLOCK, timeout=10)
50      return int(latest_block["result"]["block"]["header"]["height"])
51
52
53  latest_height = *_**
54  print(f最新のブロック番号: *_**")
55  print(f"BASE_NAME: *_**")
56  print(f保存先ディレクトリ: *_**")
57
58  for i in tqdm(range(BLOCK_COUNT), desc="Fetching blocks", unit="block"):
59      height = latest_height - i
60
61      block_header = {}
62      try:
63          block_url = f"{BASE_URL_BLOCK}?height={height}"
64          block_data = get_with_retry(block_url, timeout=10)
65          block_header = (

```

```

66         block_data.get("result", {})
67         .get("block", {})
68         .get("header", {})
69     )
70 except requests.exceptions.RequestException as e:
71     print(f" Failed to fetch block header for height {height}: {e}")
72
73     block_results = {}
74     try:
75         block_results_url = f"{BASE_URL_BLOCK_RESULTS}?height={height}"
76         block_results_data = get_with_retry(block_results_url, timeout=10)
77         block_results = block_results_data.get("result", {})
78     except requests.exceptions.RequestException as e:
79         print(f" Failed to fetch block results for height {height}: {e}")
80
81     block_validators = []
82
83     for page in range(1, TOTAL_PAGES + 1):
84         url = f"{BASE_URL_VALIDATORS}?height={height}&per_page={PER_PAGE}&page={page}"
85         retries = 0
86
87         while retries < RETRY_LIMIT:
88             try:
89                 response = requests.get(url, headers=headers, timeout=10)
90                 response.raise_for_status()
91
92                 data = *_.json()
93                 result = data.get("result")
94
95                 if not result or not isinstance(result, dict) or "validators" not in
↪ result:
96                     break
97
98                 validators = result["validators"]
99
100                 if not validators:
101                     break
102
103                 block_validators.extend(validators)
104                 break
105
106             except requests.exceptions.RequestException:
107                 retries += 1
108                 time.sleep(SLEEP_TIME)
109
110             if retries == RETRY_LIMIT:
111                 print(f" Failed to fetch validator page {page} after {RETRY_LIMIT} attempts
↪ . Skipping.")
112
113     if block_header or block_results or block_validators:
114         output = {
115             "base_url": *_,
116             "blockheader": *_,
117             "blockresults": *_,
118             "validators": *_
119         }
120         filename = os.path.join(SAVE_DIR, f"{BASE_NAME}_BlockNum_{height}.json")

```

```

121         with open(filename, "w", encoding="utf-8") as f:
122             json.dump(output, f, indent=4, ensure_ascii=False)

```

このスクリプトにおいては、RPC エンドポイント、取得ブロック数、保存先ディレクトリなどを実験目的に応じて設定する必要がある。特に BASE_URL_BLOCK, BASE_URL_VALIDATORS, BLOCK_COUNT は、各自の対象チェーンや取得範囲に応じて適切に変更すること。

```

(venv) admin-y@LAPTOP-GQE54E1E:~/tmp/manuscripts/EXBC/scripts$ python3 1-1_block-fetch.py
最新のブロック番号: 85421397
BASE_NAME: dydx
保存先ディレクトリ: current_dydx
Fetching blocks: 100% | 5000/5000 [1:45:40<00:00, 1.27s/block]

```

図 3 ブロックデータ取得処理の結果

Day1: 必須課題 1-1

ブロックデータを取得するソースコードの動作を説明し、取得時間や取得データサイズについて考察し、報告せよ。

3.4 ブロックチェーン分析スクリプト

保存済みの JSON 群を対象として、各ブロックの基本情報、バリデータ数、Voting Power, Proposer Priorityなどを抽出し、CSV形式で整理する分析スクリプトをソースコード 22 に示す。このスクリプトでは、各 JSON ファイルを 1 ブロック単位で解析し、さらに前ブロックとの比較によって、Proposer Address と Proposer Priority の関係も調べる。最終的には、集約結果を block_analysis.csv として保存する。

ソースコード 22 ブロック分析スクリプト (1-2)

```

1  import os
2  import requests
3  import json
4  import time
5  import re
6  from tqdm import tqdm
7  from urllib.parse import urlparse
8
9  BASE_URL = "*_*_"
10
11  BASE_URL_BLOCK = "*_*_" + "/block"
12  BASE_URL_BLOCK_RESULTS = "*_*_" + "/block_results"
13  BASE_URL_VALIDATORS = "*_*_" + "/validators"
14
15  PER_PAGE = 100
16  TOTAL_PAGES = 1
17  RETRY_LIMIT = 100
18  SLEEP_TIME = 1
19  BLOCK_COUNT = "*_*_"
20
21  parsed_base = urlparse(BASE_URL)
22  hostname = parsed_base.hostname or ""
23

```

```

24 m = re.match(r"^([a-zA-Z0-9\-\+)-rpc", hostname)
25 BASE_NAME = m.group(1) if m else hostname.split(".")[0]
26
27 SAVE_DIR = *__*
28
29 os.makedirs(SAVE_DIR, exist_ok=True)
30
31 headers = {"User-Agent": "Mozilla/5.0"}
32
33
34 def get_with_retry(url, timeout=10, retry_limit=RETRY_LIMIT, sleep_time=SLEEP_TIME):
35     retries = 0
36
37     while retries < retry_limit:
38         try:
39             response = requests.get(url, headers=headers, timeout=timeout)
40             response.raise_for_status()
41             return response.json()
42
43         except requests.exceptions.RequestException as e:
44             retries += 1
45
46             if retries >= retry_limit:
47                 raise e
48
49             time.sleep(sleep_time)
50
51
52 def get_latest_height():
53     latest_block = get_with_retry(BASE_URL_BLOCK, timeout=10)
54
55     return int(
56         latest_block["result"]["block"]["header"]["height"]
57     )
58
59
60 latest_height = *__*
61
62 print(f最新のブロック番号: *__*)
63 print(fBASE_NAME: *__*)
64 print(f保存先ディレクトリ: *__*)
65
66 for i in tqdm(range(BLOCK_COUNT), desc="Fetching blocks", unit="block"):
67     height = latest_height - i
68
69     block_header = {}
70     block_last_commit = {}
71
72     try:
73         block_url = f"{BASE_URL_BLOCK}?height={height}"
74         block_data = get_with_retry(block_url, timeout=10)
75
76         block = (
77             block_data.get("result", {})
78             .get("block", {})
79         )
80

```

```

81     block_header = block.get("header", {})
82     block_last_commit = block.get("last_commit", {})
83
84     except requests.exceptions.RequestException as e:
85         print(f" Failed to fetch block header/last_commit for height {height}: {e}")
86
87     block_results = {}
88
89     try:
90         block_results_url = f"{BASE_URL_BLOCK_RESULTS}?height={height}"
91         block_results_data = get_with_retry(block_results_url, timeout=10)
92         block_results = block_results_data.get("result", {})
93
94     except requests.exceptions.RequestException as e:
95         print(f" Failed to fetch block results for height {height}: {e}")
96
97     block_validators = []
98
99     for page in range(1, TOTAL_PAGES + 1):
100         url = f"{BASE_URL_VALIDATORS}?height={height}&per_page={PER_PAGE}&page={page}"
101         retries = 0
102
103         while retries < RETRY_LIMIT:
104             try:
105                 response = requests.get(url, headers=headers, timeout=10)
106                 response.raise_for_status()
107
108                 data = **_
109                 result = data.get("result")
110
111                 if not result or not isinstance(result, dict) or "validators" not in
↪ result:
112                     break
113
114                 validators = result["validators"]
115
116                 if not validators:
117                     break
118
119                 block_validators.extend(validators)
120                 break
121
122             except requests.exceptions.RequestException:
123                 retries += 1
124                 time.sleep(SLEEP_TIME)
125
126         if retries == RETRY_LIMIT:
127             print(f" Failed to fetch validator page {page} after {RETRY_LIMIT} attempts
↪ . Skipping.")
128
129     if block_header or block_last_commit or block_results or block_validators:
130         output = {
131             "base_url": **_,
132             "blockheader": **_,
133             "last_commit": block_last_commit,
134             "blockresults": **_,
135             "validators": **_

```

```

136     }
137
138     filename = os.path.join(
139         SAVE_DIR,
140         f"{BASE_NAME}_BlockNum_{height}.json"
141     )
142
143     with open(filename, "w", encoding="utf-8") as f:
144         json.dump(output, f, indent=4, ensure_ascii=False)

```

この分析スクリプトにより、個々のブロックに含まれる情報を表形式に整理できるため、後続の統計処理や可視化が容易になる。また、前ブロックの Proposer Priority と現ブロックの Proposer Address を比較することで、提案者選出の傾向に関する基礎的な観察も可能となる。

Day1: 必須課題 1-2

1-1 で取得したブロックデータを用いて、ソースコード 1-2 を実行せよ。またソースコードの動作を説明し、取得時間や取得データサイズについて、分析結果を確認したうえで報告せよ。

3.5 バリデータ分析スクリプト

バリデータ (Validator) とは、ブロックチェーンの合意形成に参加し、ブロック提案や署名を行うノードである。Cosmos / CometBFT 系チェーンでは、各ブロック高におけるバリデータ集合、Voting Power、Proposer Priority などを確認することで、合意形成に関わるノード群の変化を分析できる。

本節では、保存済み JSON 群を対象として、バリデータ集合の出現状況、Voting Power の推移、ブロック高ごとのバリデータ数、および連続ブロック間の差分を分析するスクリプトをソースコード 23 に示す。

ソースコード 23 バリデータ分析スクリプト (1-3)

```

1  #!/usr/bin/env python3
2  # -*- coding: utf-8 -*-
3
4  import os
5  import re
6  import json
7  import csv
8  from collections import defaultdict
9
10 import matplotlib.pyplot as *
11 from matplotlib.ticker import ScalarFormatter
12
13 INPUT_DIR = "current_*)"
14 OUT_DIR = "out_*)_validator_analysis"
15
16 PROPOSER_PRIORITY_PLAIN_FIG_WIDTH = 80
17 PROPOSER_PRIORITY_PLAIN_FIG_HEIGHT = 8
18
19
20 def extract_height_from_filename(filename: str):
21     m = re.search(r"BlockNum_(\d+)\.json$", filename)
22     if not m:
23         return None

```

```

24     return int(m.group(1))
25
26
27 def safe_int(value, default=0):
28     try:
29         return int(value)
30     except Exception:
31         return default
32
33
34 def load_validator_files(input_dir: str):
35     rows = []
36
37     for name in os.listdir(input_dir):
38         if not name.endswith(".json"):
39             continue
40
41         path = os.path.join(input_dir, name)
42         height = extract_height_from_filename(name)
43
44         if height is None:
45             continue
46
47         try:
48             with open(path, "*_*", encoding="utf-8") as f:
49                 data = json.load(*_*)
50
51                 *_*.append((height, path, data))
52
53         except Exception as e:
54             print(f"[WARN] failed to read {path}: {e}")
55
56     rows.sort(key=lambda x: x[0])
57     return rows
58
59
60 def normalize_validators(*_*):
61     out = {}
62
63     for v in validators:
64         address = v.get("address", "")
65
66         if not address:
67             continue
68
69         pub_key = v.get("pub_key", {})
70         pub_key_type = pub_key.get("type", "") if isinstance(pub_key, dict) else ""
71         pub_key_value = pub_key.get("value", "") if isinstance(pub_key, dict) else ""
72
73         out[address] = {
74             "address": address,
75             "pub_key_type": pub_key_type,
76             "pub_key_value": pub_key_value,
77             "voting_power": safe_int(v.get("voting_power", 0)),
78             "proposer_priority": safe_int(v.get("proposer_priority", 0)),
79         }
80

```



```

81     return out
82
83
84 def summarize_validator_presence(*_):
85     stats = defaultdict(lambda: {
86         "first_height": None,
87         "last_height": None,
88         "appearance_count": 0,
89         "min_voting_power": None,
90         "max_voting_power": None,
91         "sum_voting_power": 0,
92         "pub_key_type": "",
93         "pub_key_value": "",
94     })
95
96     for height, _, data in blocks:
97         validators = normalize_validators(data.get("validators", []))
98
99         for addr, info in validators.items():
100             st = stats[addr]
101
102             if st["first_height"] is None:
103                 st["first_height"] = height
104
105             st["last_height"] = height
106             st["appearance_count"] += 1
107
108             vp = info["voting_power"]
109
110             if st["min_voting_power"] is None or vp < st["min_voting_power"]:
111                 st["min_voting_power"] = vp
112
113             if st["max_voting_power"] is None or vp > st["max_voting_power"]:
114                 st["max_voting_power"] = vp
115
116             st["sum_voting_power"] += vp
117             st["pub_key_type"] = info["pub_key_type"]
118             st["pub_key_value"] = info["pub_key_value"]
119
120     return stats
121
122
123 def analyze_per_height(blocks):
124     per_height = []
125
126     for height, path, data in blocks:
127         validators = normalize_validators(data.get("validators", []))
128         total_voting_power = sum(v["voting_power"] for v in validators.values())
129
130         per_height.append({
131             "height": height,
132             "file": os.path.basename(path),
133             "validator_count": len(validators),
134             "total_voting_power": total_voting_power,
135         })
136
137     return per_height

```

```

138
139
140 def analyze_diffs(blocks):
141     diffs = []
142
143     prev_height = None
144     prev_validators = None
145
146     for height, _, data in blocks:
147         current_validators = normalize_validators(data.get("validators", []))
148
149         if prev_validators is not None:
150             prev_set = set(prev_validators.keys())
151             curr_set = set(current_validators.keys())
152
153             added = sorted(curr_set - prev_set)
154             removed = sorted(prev_set - curr_set)
155             common = sorted(prev_set & curr_set)
156
157             power_changed = []
158             priority_changed = []
159
160             for addr in common:
161                 prev_v = prev_validators[addr]
162                 curr_v = current_validators[addr]
163
164                 if prev_v["voting_power"] != curr_v["voting_power"]:
165                     power_changed.append({
166                         "address": addr,
167                         "prev_voting_power": prev_v["voting_power"],
168                         "curr_voting_power": curr_v["voting_power"],
169                     })
170
171                 if prev_v["proposer_priority"] != curr_v["proposer_priority"]:
172                     priority_changed.append({
173                         "address": addr,
174                         "prev_proposer_priority": prev_v["proposer_priority"],
175                         "curr_proposer_priority": curr_v["proposer_priority"],
176                     })
177
178             diffs.append({
179                 "prev_height": prev_height,
180                 "curr_height": height,
181                 "added_count": len(added),
182                 "removed_count": len(removed),
183                 "power_changed_count": len(power_changed),
184                 "priority_changed_count": len(priority_changed),
185                 "added": added,
186                 "removed": removed,
187                 "power_changed": power_changed,
188                 "priority_changed": priority_changed,
189             })
190
191     prev_height = height
192     prev_validators = current_validators
193
194     return diffs

```

```

195
196
197 def build_validator_power_timeseries(blocks):
198     heights = [height for height, _, _ in blocks]
199     address_to_series = defaultdict(list)
200
201     all_addresses = set()
202     normalized_by_height = []
203
204     for _, _, data in blocks:
205         validators = normalize_validators(data.get("validators", []))
206         normalized_by_height.append(validators)
207         all_addresses.update(validators.keys())
208
209     all_addresses = sorted(all_addresses)
210
211     for addr in all_addresses:
212         address_to_series[addr] = []
213
214     for validators in normalized_by_height:
215         current_addresses = set(validators.keys())
216
217         for addr in all_addresses:
218             if addr in current_addresses:
219                 address_to_series[addr].append(validators[addr]["voting_power"])
220             else:
221                 address_to_series[addr].append(None)
222
223     return heights, address_to_series
224
225
226 def build_validator_priority_timeseries(blocks):
227     heights = [height for height, _, _ in blocks]
228     address_to_series = defaultdict(list)
229
230     all_addresses = set()
231     normalized_by_height = []
232
233     for _, _, data in blocks:
234         validators = normalize_validators(data.get("validators", []))
235         normalized_by_height.append(validators)
236         all_addresses.update(validators.keys())
237
238     all_addresses = sorted(all_addresses)
239
240     for addr in all_addresses:
241         address_to_series[addr] = []
242
243     for validators in normalized_by_height:
244         current_addresses = set(validators.keys())
245
246         for addr in all_addresses:
247             if addr in current_addresses:
248                 address_to_series[addr].append(validators[addr]["proposer_priority"])
249             else:
250                 address_to_series[addr].append(None)
251

```

```

252     return heights, address_to_series
253
254
255 def write_presence_csv(stats, out_csv):
256     with open(out_csv, "w", newline="", encoding="utf-8") as f:
257         writer = csv.writer(f)
258
259         writer.writerow([
260             "address",
261             "first_height",
262             "last_height",
263             "appearance_count",
264             "avg_voting_power",
265             "min_voting_power",
266             "max_voting_power",
267             "pub_key_type",
268             "pub_key_value",
269         ])
270
271         for addr, st in sorted(stats.items(), key=lambda x: (-x[1]["appearance_count"],
↪ x[0])):
272             avg_vp = st["sum_voting_power"] / st["appearance_count"] if st["
↪ appearance_count"] else 0
273
274             writer.writerow([
275                 addr,
276                 st["first_height"],
277                 st["last_height"],
278                 st["appearance_count"],
279                 f"{avg_vp:.2f}",
280                 st["min_voting_power"],
281                 st["max_voting_power"],
282                 st["pub_key_type"],
283                 st["pub_key_value"],
284             ])
285
286
287 def write_per_height_csv(per_height, *_):
288     with open(out_csv, "w", newline="", encoding="utf-8") as f:
289         writer = csv.writer(f)
290
291         writer.writerow([
292             "height",
293             "file",
294             "validator_count",
295             "total_voting_power",
296         ])
297
298         for row in per_height:
299             writer.writerow([
300                 row["height"],
301                 row["file"],
302                 row["validator_count"],
303                 row["total_voting_power"],
304             ])
305
306

```

```

307 def write_diff_summary_csv(diffs, out_csv):
308     with open(out_csv, "w", newline="", encoding="utf-8") as f:
309         writer = csv.writer(f)
310
311         writer.writerow([
312             "prev_height",
313             "curr_height",
314             "added_count",
315             "removed_count",
316             "power_changed_count",
317             "priority_changed_count",
318         ])
319
320         for d in diffs:
321             writer.writerow([
322                 d["prev_height"],
323                 d["curr_height"],
324                 d["added_count"],
325                 d["removed_count"],
326                 d["power_changed_count"],
327                 d["priority_changed_count"],
328             ])
329
330
331 def write_diff_details_json(diffs, out_json):
332     with open(out_json, "w", encoding="utf-8") as f:
333         json.dump(diffs, f, indent=2, ensure_ascii=False)
334
335
336 def shorten_address(addr: str, head=8, tail=6):
337     if len(addr) <= head + tail + 3:
338         return addr
339     return f"{addr[:head]}...{addr[-tail:]}"
340
341
342 def apply_axis_format(use_scientific: bool, axis="both"):
343     ax = plt.gca()
344
345     if use_scientific:
346         formatter_x = ScalarFormatter(useMathText=False)
347         formatter_y = ScalarFormatter(useMathText=False)
348
349         formatter_x.set_scientific(True)
350         formatter_y.set_scientific(True)
351
352         formatter_x.set_powerlimits((0, 0))
353         formatter_y.set_powerlimits((0, 0))
354
355         if axis in ("x", "both"):
356             ax.xaxis.set_major_formatter(formatter_x)
357             ax.ticklabel_format(axis="x", style="sci", scilimits=(0, 0))
358
359         if axis in ("y", "both"):
360             ax.yaxis.set_major_formatter(formatter_y)
361             ax.ticklabel_format(axis="y", style="sci", scilimits=(0, 0))
362
363     else:

```

```

364     formatter_x = ScalarFormatter(useOffset=False)
365     formatter_y = ScalarFormatter(useOffset=False)
366
367     formatter_x.set_scientific(False)
368     formatter_y.set_scientific(False)
369
370     if axis in ("x", "both"):
371         ax.xaxis.set_major_formatter(formatter_x)
372         ax.ticklabel_format(axis="x", style="plain", useOffset=False)
373
374     if axis in ("y", "both"):
375         ax.yaxis.set_major_formatter(formatter_y)
376         ax.ticklabel_format(axis="y", style="plain", useOffset=False)
377
378
379 def extract_proposer_maps(blocks):
380     *__* = {}
381     *__* = {}
382
383     for height, _, data in blocks:
384         blockheader = data.get("blockheader", {})
385         proposer_address = blockheader.get("proposer_address", "")
386         proposer_by_height[height] = proposer_address
387         validators_by_height[height] = normalize_validators(data.get("validators", []))
388
389     return proposer_by_height, validators_by_height
390
391
392 def plot_validator_count(per_height, out_path, use_scientific=False):
393     heights = [x["height"] for x in per_height]
394     counts = [x["validator_count"] for x in per_height]
395
396     plt.figure(figsize=(12, 6))
397     plt.plot(heights, counts, marker="o", markersize=2, linewidth=1)
398     plt.xlabel("Height")
399     plt.ylabel("Validator Count")
400     plt.title("Validator Count over Height")
401     plt.grid(True)
402     apply_axis_format(use_scientific, axis="x")
403     plt.tight_layout()
404     plt.savefig(out_path, dpi=200)
405     plt.close()
406
407
408 def plot_validator_set_changes(diffs, out_path, use_scientific=False):
409     if not diffs:
410         return
411
412     heights = [x["curr_height"] for x in diffs]
413     added = [x["added_count"] for x in diffs]
414     removed = [x["removed_count"] for x in diffs]
415     power_changed = [x["power_changed_count"] for x in diffs]
416
417     plt.figure(figsize=(12, 6))
418     plt.plot(heights, added, label="added_count", marker="o", markersize=2, linewidth=1)
419     plt.plot(heights, removed, label="removed_count", marker="o", markersize=2,
↪ linewidth=1)

```

```

420     plt.plot(heights, power_changed, label="power_changed_count", marker="o", markersize
↪ =2, linewidth=1)
421     plt.xlabel("Current Height")
422     plt.ylabel("Count")
423     plt.title("Validator Set Changes over Height")
424     plt.grid(True)
425     plt.legend()
426     apply_axis_format(use_scientific, axis="x")
427     plt.tight_layout()
428     plt.savefig(out_path, dpi=200)
429     plt.close()
430
431
432 def plot_priority_changed_count(diffs, out_path, use_scientific=False):
433     if not diffs:
434         return
435
436     heights = [x["curr_height"] for x in diffs]
437     priority_changed = [x["priority_changed_count"] for x in diffs]
438
439     plt.figure(figsize=(12, 6))
440     plt.plot(heights, priority_changed, marker="o", markersize=2, linewidth=1)
441     plt.xlabel("Current Height")
442     plt.ylabel("Priority Changed Count")
443     plt.title("Proposer Priority Changes over Height")
444     plt.grid(True)
445     apply_axis_format(use_scientific, axis="x")
446     plt.tight_layout()
447     plt.savefig(out_path, dpi=200)
448     plt.close()
449
450
451 def plot_all_validators_voting_power(
452     blocks,
453     stats,
454     out_path,
455     use_scientific=True,
456     fig_width=16,
457     fig_height=8
458 ):
459     heights, series = build_validator_power_timeseries(blocks)
460
461     ranked_addresses = [
462         addr for addr, _ in sorted(
463             stats.items(),
464             key=lambda x: (-x[1]["appearance_count"], x[0])
465         )
466     ]
467
468     if not ranked_addresses:
469         return
470
471     proposer_by_height, validators_by_height = extract_proposer_maps(blocks)
472
473     plt.figure(figsize=(fig_width, fig_height))
474
475     line_color_by_addr = {}

```

```

476
477     for addr in ranked_addresses:
478         values = series[addr]
479         line, = plt.plot(
480             heights,
481             values,
482             label=shorten_address(addr),
483             linewidth=1
484         )
485         line_color_by_addr[addr] = line.get_color()
486
487     for addr in ranked_addresses:
488         marker_heights = []
489         marker_values = []
490
491         for height in heights:
492             if proposer_by_height.get(height, "") != addr:
493                 continue
494
495             validators = validators_by_height.get(height, {})
496             if addr not in validators:
497                 continue
498
499             marker_heights.append(height)
500             marker_values.append(validators[addr]["voting_power"])
501
502         if marker_heights:
503             plt.scatter(
504                 marker_heights,
505                 marker_values,
506                 marker="x",
507                 s=35,
508                 linewidths=1.2,
509                 color=line_color_by_addr.get(addr),
510                 label=None,
511                 zorder=5
512             )
513
514     plt.xlabel("Height")
515     plt.ylabel("Voting Power")
516     plt.title("Voting Power over Height")
517     plt.grid(True)
518     plt.legend(
519         fontsize=6,
520         loc="center left",
521         bbox_to_anchor=(1.02, 0.5),
522         borderaxespad=0.0,
523         ncol=1
524     )
525     apply_axis_format(use_scientific, axis="both")
526     plt.tight_layout(rect=[0, 0, 0.82, 1])
527     plt.savefig(out_path, dpi=200, bbox_inches="tight")
528     plt.close()
529
530
531 def plot_all_validators_proposer_priority(
532     blocks,

```



```

533     stats,
534     out_path,
535     use_scientific=True,
536     fig_width=16,
537     fig_height=8
538 ):
539     heights, series = build_validator_priority_timeseries(blocks)
540
541     ranked_addresses = [
542         addr for addr, _ in sorted(
543             stats.items(),
544             key=lambda x: (-x[1]["appearance_count"], x[0])
545         )
546     ]
547
548     if not ranked_addresses:
549         return
550
551     proposer_by_height, validators_by_height = extract_proposer_maps(blocks)
552
553     plt.figure(figsize=(fig_width, fig_height))
554
555     line_color_by_addr = {}
556
557     for addr in ranked_addresses:
558         values = series[addr]
559         line, = plt.plot(
560             heights,
561             values,
562             label=shorten_address(addr),
563             linewidth=1
564         )
565         line_color_by_addr[addr] = line.get_color()
566
567     for addr in ranked_addresses:
568         marker_heights = []
569         marker_values = []
570
571         for height in heights:
572             if proposer_by_height.get(height, "") != addr:
573                 continue
574
575             validators = validators_by_height.get(height, {})
576             if addr not in validators:
577                 continue
578
579             marker_heights.append(height)
580             marker_values.append(validators[addr]["proposer_priority"])
581
582     if marker_heights:
583         plt.scatter(
584             marker_heights,
585             marker_values,
586             marker="x",
587             s=35,
588             linewidths=1.2,
589             color=line_color_by_addr.get(addr),

```

```

590         label=None,
591         zorder=5
592     )
593
594     plt.xlabel("Height")
595     plt.ylabel("Proposer Priority")
596     plt.title("Proposer Priority over Height")
597     plt.grid(True)
598     plt.legend(
599         fontsize=6,
600         loc="center left",
601         bbox_to_anchor=(1.02, 0.5),
602         borderaxespad=0.0,
603         ncol=1
604     )
605     apply_axis_format(use_scientific, axis="both")
606     plt.tight_layout(rect=[0, 0, 0.82, 1])
607     plt.savefig(out_path, dpi=200, bbox_inches="tight")
608     plt.close()
609
610
611 def print_summary(*_, stats, per_height, diffs):
612     print("==== Validator Analysis Summary =====")
613     print(f"blocks loaded           : {len(blocks)}")
614     print(f"unique validators          : {len(stats)}")
615
616     if per_height:
617         counts = [x["validator_count"] for x in per_height]
618         totals = [x["total_voting_power"] for x in per_height]
619
620         print(f"validator count min/max      : {min(counts)} / {max(counts)}")
621         print(f"total voting power min/max   : {min(totals)} / {max(totals)}")
622
623         added_total = sum(d["added_count"] for d in diffs)
624         removed_total = sum(d["removed_count"] for d in diffs)
625         power_changed_total = sum(d["power_changed_count"] for d in diffs)
626         priority_changed_total = sum(d["priority_changed_count"] for d in diffs)
627
628         print(f"total added events           : {added_total}")
629         print(f"total removed events        : {removed_total}")
630         print(f"total power changes          : {power_changed_total}")
631         print(f"total priority changes       : {priority_changed_total}")
632         print()
633
634         print("Top validators by appearance_count")
635
636         ranked = sorted(stats.items(), key=lambda x: (-x[1]["appearance_count"], x[0]))[:10]
637
638         for i, (addr, st) in enumerate(ranked, 1):
639             avg_vp = st["sum_voting_power"] / st["appearance_count"] if st["appearance_count"]
640             ↪ 0
641
642             print(
643                 f"{i:2d}. {addr} "
644                 f"appear={st['appearance_count']} "
645                 f"first={st['first_height']} "
646                 f"last={st['last_height']} "

```

```

646         f"avg_vp={avg_vp:.2f}"
647     )
648
649
650 def main():
651     os.makedirs(OUT_DIR, exist_ok=True)
652
653     blocks = load_validator_files(INPUT_DIR)
654
655     if not blocks:
656         print(f"[ERROR] no valid JSON files found in {INPUT_DIR}")
657         return 1
658
659     stats = summarize_validator_presence(blocks)
660     per_height = analyze_per_height(blocks)
661     diffs = analyze_diffs(blocks)
662
663     presence_csv = os.path.join(OUT_DIR, "validator_presence.csv")
664     per_height_csv = os.path.join(OUT_DIR, "validator_set_per_height.csv")
665     diff_summary_csv = os.path.join(OUT_DIR, "validator_diff_summary.csv")
666     diff_details_json = os.path.join(OUT_DIR, "validator_diff_details.json")
667
668     plot_validator_count_path = os.path.join(
669         OUT_DIR,
670         "validator_count_over_height.png"
671     )
672
673     plot_validator_changes_path = os.path.join(
674         OUT_DIR,
675         "validator_set_changes_over_height.png"
676     )
677
678     plot_priority_changed_count_path = os.path.join(
679         OUT_DIR,
680         "priority_changed_count_over_height.png"
681     )
682
683     plot_all_validators_voting_power_scientific_path = os.path.join(
684         OUT_DIR,
685         "all_validators_voting_power_scientific.png"
686     )
687
688     plot_all_validators_voting_power_plain_path = os.path.join(
689         OUT_DIR,
690         "all_validators_voting_power_plain.png"
691     )
692
693     plot_all_validators_proposer_priority_scientific_path = os.path.join(
694         OUT_DIR,
695         "all_validators_proposer_priority_scientific.png"
696     )
697
698     plot_all_validators_proposer_priority_plain_path = os.path.join(
699         OUT_DIR,
700         "all_validators_proposer_priority_plain.png"
701     )
702

```

```

703 write_presence_csv(stats, presence_csv)
704 write_per_height_csv(per_height, per_height_csv)
705 write_diff_summary_csv(diffs, diff_summary_csv)
706 write_diff_details_json(diffs, diff_details_json)
707
708 plot_validator_count(
709     per_height,
710     plot_validator_count_path,
711     use_scientific=False
712 )
713
714 plot_validator_set_changes(
715     diffs,
716     plot_validator_changes_path,
717     use_scientific=False
718 )
719
720 plot_priority_changed_count(
721     diffs,
722     plot_priority_changed_count_path,
723     use_scientific=False
724 )
725
726 plot_all_validators_voting_power(
727     blocks,
728     stats,
729     plot_all_validators_voting_power_scientific_path,
730     use_scientific=True,
731     fig_width=16,
732     fig_height=8
733 )
734
735 plot_all_validators_voting_power(
736     blocks,
737     stats,
738     plot_all_validators_voting_power_plain_path,
739     use_scientific=False,
740     fig_width=16,
741     fig_height=8
742 )
743
744 plot_all_validators_proposer_priority(
745     blocks,
746     stats,
747     plot_all_validators_proposer_priority_scientific_path,
748     use_scientific=True,
749     fig_width=16,
750     fig_height=8
751 )
752
753 plot_all_validators_proposer_priority(
754     blocks,
755     stats,
756     plot_all_validators_proposer_priority_plain_path,
757     use_scientific=False,
758     fig_width=PROPOSER_PRIORITY_PLAIN_FIG_WIDTH,
759     fig_height=PROPOSER_PRIORITY_PLAIN_FIG_HEIGHT

```

```

760 )
761
762 print_summary(blocks, stats, per_height, diffs)
763
764 print()
765 print("==== Output Files =====")
766 print(presence_csv)
767 print(per_height_csv)
768 print(diff_summary_csv)
769 print(diff_details_json)
770 print(plot_validator_count_path)
771 print(plot_validator_changes_path)
772 print(plot_priority_changed_count_path)
773 print(plot_all_validators_voting_power_scientific_path)
774 print(plot_all_validators_voting_power_plain_path)
775 print(plot_all_validators_proposer_priority_scientific_path)
776 print(plot_all_validators_proposer_priority_plain_path)
777
778 return 0
779
780
781 if __name__ == "__main__":
782     raise SystemExit(main())

```

このスクリプトにより、ブロック高ごとのバリデータ数、Voting Power の合計、バリデータ集合の追加・削除、Voting Power や Proposer Priority の変化を分析できる。また、CSV および図として出力されるため、Proposer-Visualizer や表計算ソフトと組み合わせた分析にも利用できる。

Day1: 必須課題 1-3

ソースコード 1-3 を用いて、1-1 で取得したデータを分析、ソースコードの動作を説明し、バリデータ集合 (Validator Set) の変化、Voting Power、Proposer Priority の推移について考察し、報告せよ。

ソースコード 24 分析スクリプト (1-4)

```

1 import os
2 import re
3 import *__*
4 import pandas as *__*
5 import matplotlib.pyplot as *__*
6
7 BLOCK_ANALYSIS_CSV = "blockheader_blockresults_analysis.csv"
8 VALIDATOR_PRESENCE_CSV = "out_o*__*_validator_analysis/validator_presence.csv"
9 VALIDATOR_SET_PER_HEIGHT_CSV = "out_*__*_validator_analysis/validator_set_per_height.csv"
10 ↵ "
11 RAW_BLOCK_JSON_DIR = "current_*__*"
12
13 OUT_DIR = "out_signature_block_interval_analysis"
14
15 MERGED_CSV = "signature_block_interval_validator_merged.csv"
16
17 SCATTER_SIMPLE = "sig_span_vs_block_diff_simple.png"
18 SCATTER_BY_VALIDATOR_COUNT = "sig_span_vs_block_diff_by_validator_count.png"
19 SCATTER_BY_TOTAL_VOTING_POWER = "sig_span_vs_block_diff_by_total_voting_power.png"
20 SCATTER_PROPOSER_RANK = "proposer_rank_vs_block_diff.png"

```

```

20 SCATTER_PREV_PROPOSER_RANK = "prev_proposer_rank_vs_block_diff.png"
21 CURRENT_PROPOSER_PREV_RANK_LINE = "current_proposer_prev_rank_over_height.png"
22 CURRENT_PROPOSER_PREV_RANK_SCATTER = "current_proposer_prev_rank_vs_block_diff.png"
23
24 MAX_SIGNATURE_SPAN_SEC = 60
25 MAX_BLOCK_DIFF_SEC = None
26
27
28 def ensure_columns(df: pd.DataFrame, required_columns: list[str], name: str):
29     missing = [col for col in required_columns if col not in df.columns]
30     if missing:
31         raise ValueError(f"{name} に必要な列がありません: {missing}")
32
33
34 def extract_height_from_filename(filename: str):
35     m = re.search(r"BlockNum_(\d+)\.json$", filename)
36     if not m:
37         return None
38     return int(m.group(1))
39
40
41 def safe_int(value, default=0):
42     try:
43         return int(value)
44     except Exception:
45         return default
46
47
48 def load_data():
49     block_df = pd.read_csv(BLOCK_ANALYSIS_CSV)
50     presence_df = pd.read_csv(VALIDATOR_PRESENCE_CSV)
51     per_height_df = pd.read_csv(VALIDATOR_SET_PER_HEIGHT_CSV)
52
53     ensure_columns(
54         block_df,
55         ["height", "block_diff_sec", "sig_time_span_sec"],
56         BLOCK_ANALYSIS_CSV
57     )
58
59     ensure_columns(
60         per_height_df,
61         ["height", "validator_count", "total_voting_power"],
62         VALIDATOR_SET_PER_HEIGHT_CSV
63     )
64
65     ensure_columns(
66         presence_df,
67         ["address", "appearance_count"],
68         VALIDATOR_PRESENCE_CSV
69     )
70
71     block_df["height"] = pd.to_numeric(block_df["height"], errors="coerce")
72     block_df["block_diff_sec"] = pd.to_numeric(block_df["block_diff_sec"], errors="
↪ coerce")
73     block_df["sig_time_span_sec"] = pd.to_numeric(block_df["sig_time_span_sec"], errors
↪ ="coerce")
74

```

```

75     per_height_df["height"] = pd.to_numeric(per_height_df["height"], errors="coerce")
76     per_height_df["validator_count"] = pd.to_numeric(per_height_df["validator_count"],
↪ errors="coerce")
77     per_height_df["total_voting_power"] = pd.to_numeric(per_height_df["
↪ total_voting_power"], errors="coerce")
78
79     return block_df, presence_df, per_height_df
80
81
82 def load_raw_block_rows(raw_json_dir: str):
83     rows = []
84
85     if not os.path.isdir(raw_json_dir):
86         print(f"[WARN] raw JSON directory not found: {raw_json_dir}")
87         return rows
88
89     for filename in sorted(os.listdir(raw_json_dir)):
90         if not filename.endswith(".json"):
91             continue
92
93         height = extract_height_from_filename(filename)
94         if height is None:
95             continue
96
97         path = os.path.join(raw_json_dir, filename)
98
99         try:
100             with open(path, "r", encoding="utf-8") as f:
101                 data = json.load(f)
102         except Exception as e:
103             print(f"[WARN] failed to read {path}: {e}")
104             continue
105
106         blockheader = data.get("blockheader", {}) or {}
107         validators = data.get("validators", []) or []
108
109         proposer_address = blockheader.get("proposer_address")
110
111         normalized_validators = []
112
113         for v in validators:
114             if not isinstance(v, dict):
115                 continue
116
117             address = v.get("address")
118             if not address:
119                 continue
120
121             normalized_validators.append({
122                 "address": address,
123                 "proposer_priority": safe_int(v.get("proposer_priority", 0)),
124                 "voting_power": safe_int(v.get("voting_power", 0)),
125             })
126
127         rows.append({
128             "height": *__,
129             "proposer_address": proposer*__,

```

```

130         "validators": normalized_validators,
131         "validator_count_from_json": len(normalized_validators),
132     })
133
134     rows.sort(key=lambda x: x["height"])
135     return rows
136
137
138 def calc_proposer_rank_for_block(proposer_address, validators):
139     if not proposer_address or not validators:
140         return {
141             "proposer_priority_rank": None,
142             "proposer_voting_power_rank": None,
143             "proposer_priority_value": None,
144             "proposer_voting_power": None,
145         }
146
147     sorted_by_priority = sorted(
148         validators,
149         key=lambda x: (-x["proposer_priority"], -x["voting_power"], x["address"])
150     )
151
152     sorted_by_voting_power = sorted(
153         validators,
154         key=lambda x: (-x["voting_power"], -x["proposer_priority"], x["address"])
155     )
156
157     proposer_priority_rank = None
158     proposer_voting_power_rank = None
159     proposer_priority_value = None
160     proposer_voting_power = None
161
162     for idx, row in enumerate(sorted_by_priority, start=1):
163         if row["address"] == proposer_address:
164             proposer_priority_rank = idx
165             proposer_priority_value = row["proposer_priority"]
166             proposer_voting_power = row["voting_power"]
167             break
168
169     for idx, row in enumerate(sorted_by_voting_power, start=1):
170         if row["address"] == proposer_address:
171             proposer_voting_power_rank = idx
172             if proposer_voting_power is None:
173                 proposer_voting_power = row["voting_power"]
174             if proposer_priority_value is None:
175                 proposer_priority_value = row["*_*"]
176             break
177
178     return {
179         "proposer_priority_rank": *_*,
180         "proposer_voting_power_rank": proposer_voting_power_rank,
181         "proposer_priority_value": proposer_priority_value,
182         "proposer_voting_power": proposer_voting_power,
183     }
184
185
186 def calc_current_proposer_values_in_prev_block(proposer_address, prev_validators):

```



```

187     if not proposer_address or not prev_validators:
188         return {
189             "current_proposer_priority_in_prev_block": None,
190             "current_proposer_rank_in_prev_block": None,
191             "current_proposer_voting_power_in_prev_block": None,
192         }
193
194     prev_sorted_by_priority = sorted(
195         prev_validators,
196         key=lambda x: (-x["proposer_priority"], -x["voting_power"], x["address"])
197     )
198
199     current_proposer_priority_in_prev_block = None
200     current_proposer_rank_in_prev_block = None
201     current_proposer_voting_power_in_prev_block = None
202
203     for idx, row in enumerate(prev_sorted_by_priority, start=1):
204         if row["address"] == proposer_address:
205             current_proposer_priority_in_prev_block = row["proposer_priority"]
206             current_proposer_rank_in_prev_block = idx
207             current_proposer_voting_power_in_prev_block = row["voting_power"]
208             break
209
210     return {
211         "current_proposer_priority_in_prev_block":
↪ current_proposer_priority_in_prev_block,
212         "current_proposer_rank_in_prev_block": current_proposer_rank_in_prev_block,
213         "current_proposer_voting_power_in_prev_block":
↪ current_proposer_voting_power_in_prev_block,
214     }
215
216
217 def build_proposer_rank_df(raw_json_dir: str) -> pd.DataFrame:
218     block_rows = load_raw_block_rows(raw_json_dir)
219
220     if not block_rows:
221         return pd.DataFrame()
222
223     validators_by_height = {
224         row["height"]: row["validators"]
225         for row in block_rows
226     }
227
228     proposer_by_height = {
229         row["height"]: row["proposer_address"]
230         for row in block_rows
231     }
232
233     records = []
234
235     for row in block_rows:
236         height = row["height"]
237         proposer_address = row["*_**"]
238         validators = row["validators"]
239         prev_height = height - 1
240
241         proposer_rank_info = calc_proposer_rank_for_block(

```

```

242         proposer_address,
243         validators
244     )
245
246     current_proposer_prev_info = calc_current_proposer_values_in_prev_block(
247         proposer_address,
248         validators_by_height.get(prev_height, [])
249     )
250
251     prev_proposer_address = proposer_by_height.get(prev_height)
252     prev_proposer_rank_info = calc_proposer_rank_for_block(
253         prev_proposer_address,
254         validators_by_height.get(prev_height, [])
255     )
256
257     records.append({
258         "height": height,
259         "proposer_address": proposer_address,
260         "proposer_priority_rank": proposer_rank_info["proposer_priority_rank"],
261         "proposer_voting_power_rank": proposer_rank_info["proposer_voting_power_rank"]
↪ "],
262         "proposer_priority_value": proposer_rank_info["proposer_priority_value"],
263         "proposer_voting_power": proposer_rank_info["proposer_voting_power"],
264         "validator_count_from_json": row["validator_count_from_json"],
265         "prev_height": prev_height,
266         "prev_proposer_address": prev_proposer_address,
267         "prev_proposer_priority_rank": prev_proposer_rank_info["
↪ proposer_priority_rank"],
268         "prev_proposer_voting_power_rank": prev_proposer_rank_info["
↪ proposer_voting_power_rank"],
269         "prev_proposer_priority_value": prev_proposer_rank_info["
↪ proposer_priority_value"],
270         "prev_proposer_voting_power": prev_proposer_rank_info["proposer_voting_power
↪ "],
271         "current_proposer_priority_in_prev_block": current_proposer_prev_info["
↪ current_proposer_priority_in_prev_block"],
272         "current_proposer_rank_in_prev_block": current_proposer_prev_info["
↪ current_proposer_rank_in_prev_block"],
273         "current_proposer_voting_power_in_prev_block": current_proposer_prev_info["
↪ current_proposer_voting_power_in_prev_block"],
274     })
275
276     df = pd.DataFrame(records)
277
278     numeric_cols = [
279         "height",
280         "proposer_priority_rank",
281         "proposer_voting_power_rank",
282         "proposer_priority_value",
283         "proposer_voting_power",
284         "validator_count_from_json",
285         "prev_height",
286         "prev_proposer_priority_rank",
287         "prev_proposer_voting_power_rank",
288         "prev_proposer_priority_value",
289         "prev_proposer_voting_power",
290         "current_proposer_priority_in_prev_block",

```

```

291     "current_proposer_rank_in_prev_block",
292     "current_proposer_voting_power_in_prev_block",
293 ]
294
295 for col in numeric_cols:
296     if col in df.columns:
297         df[col] = pd.to_numeric(df[col], errors="coerce")
298
299 df = df.sort_values("height").reset_index(drop=True)
300
301 return df
302
303
304 def build_merged_df(
305     block_df: pd.DataFrame,
306     per_height_df: pd.DataFrame,
307     proposer_rank_df: pd.DataFrame
308 ) -> pd.DataFrame:
309     use_block_cols = [
310         "height",
311         "timestamp",
312         "block_diff_sec",
313         "sig_time_span_sec",
314         "signature_count",
315         "num_transactions",
316         "proposer_address",
317     ]
318
319     use_block_cols = [col for col in use_block_cols if col in block_df.columns]
320
321     use_per_height_cols = [
322         "height",
323         "validator_count",
324         "total_voting_power",
325     ]
326
327     merged = pd.merge(
328         block_df[use_block_cols],
329         per_height_df[use_per_height_cols],
330         on="height",
331         how="left"
332     )
333
334     if not proposer_rank_df.empty:
335         proposer_use_cols = [
336             "height",
337             "proposer_priority_rank",
338             "proposer_voting_power_rank",
339             "proposer_priority_value",
340             "proposer_voting_power",
341             "validator_count_from_json",
342             "prev_height",
343             "prev_proposer_address",
344             "prev_proposer_priority_rank",
345             "prev_proposer_voting_power_rank",
346             "prev_proposer_priority_value",
347             "prev_proposer_voting_power",

```

```

348         "current_proposer_priority_in_prev_block",
349         "current_proposer_rank_in_prev_block",
350         "current_proposer_voting_power_in_prev_block",
351     ]
352
353     proposer_use_cols = [
354         col for col in proposer_use_cols
355         if *col in proposer_rank_df.columns
356     ]
357
358     merged = pd.merge(
359         merged,
360         proposer_rank_df[proposer_use_cols],
361         on="height",
362         how="left"
363     )
364
365     merged = merged.dropna(subset=["block_diff_sec", "sig_time_span_sec"])
366
367     if MAX_SIGNATURE_SPAN_SEC is not None:
368         merged = merged[merged["sig_time_span_sec"] <= MAX_SIGNATURE_SPAN_SEC]
369
370     if MAX_BLOCK_DIFF_SEC is not None:
371         merged = merged[merged["block_diff_sec"] <= MAX_BLOCK_DIFF_SEC]
372
373     merged = merged.sort_values("height").reset_index(drop=True)
374
375     return merged
376
377
378 def plot_simple_scatter(df: pd.DataFrame, out_path: str):
379     if df.empty:
380         print("散布図を作成できません (": 有効なデータがありません。)")
381         return
382
383     plt.figure(figsize=(10, 7))
384     plt.scatter(
385         df["sig_time_span_sec"],
386         df["block_diff_sec"],
387         s=25,
388         alpha=0.7
389     )
390
391     plt.xlabel("Validator Signature Time Span per Block (seconds)")
392     plt.ylabel("Block Time Difference (seconds)")
393     plt.title("Validator Signature Time Span vs Block Time Difference")
394     plt.grid(True)
395     plt.tight_layout()
396     plt.savefig(out_path, dpi=200)
397     plt.close()
398
399     print(f"保存しました: {out_path}")
400
401
402 def plot_scatter_by_validator_count(df: pd.DataFrame, out_path: str):
403     if df.empty:
404         print("validator_count 散布図を作成できません: 有効なデータがありません。")

```

```

405         return
406
407     if "validator_count" not in df.columns or df["validator_count"].dropna().empty:
408         print("validator_count がないため、散布図を作成できません。")
409         return
410
411     plt.figure(figsize=(10, 7))
412     scatter = plt.scatter(
413         df["sig_time_span_sec"],
414         df["block_diff_sec"],
415         c=df["validator_count"],
416         s=25,
417         alpha=0.75
418     )
419
420     plt.xlabel("Validator Signature Time Span per Block (seconds)")
421     plt.ylabel("Block Time Difference (seconds)")
422     plt.title("Validator Signature Time Span vs Block Time Difference")
423     plt.grid(True)
424
425     cbar = plt.colorbar(scatter)
426     cbar.set_label("Validator Count")
427
428     plt.tight_layout()
429     plt.savefig(out_path, dpi=200)
430     plt.close()
431
432     print(f保存しました: {out_path}")
433
434
435 def plot_scatter_by_total_voting_power(df: pd.DataFrame, out_path: str):
436     if df.empty:
437         *__(f"total_voting_power 散布図を作成できません: 有効なデータがありません。")
438         return
439
440     if "total_voting_power" not in df.columns or df["total_voting_power"].dropna().empty
↪ :
441         print("total_voting_power がないため、散布図を作成できません。")
442         return
443
444     plt.figure(figsize=(10, 7))
445     scatter = plt.scatter(
446         df["sig_time_span_sec"],
447         df["block_diff_sec"],
448         c=df["total_voting_power"],
449         s=25,
450         alpha=0.75
451     )
452
453     plt.xlabel("Validator Signature Time Span per Block (seconds)")
454     plt.ylabel("Block Time Difference (seconds)")
455     plt.*__(f"Validator Signature Time Span vs Block Time Difference")
456     plt.grid(True)
457
458     cbar = plt.colorbar(scatter)
459     cbar.set_label("Total Voting Power")
460

```

```

461 plt.tight_layout()
462 plt.savefig(out_path, dpi=200)
463 plt.*_*( )
464
465 print(f保存しました": {out_path}")
466
467
468 def plot_proposer_rank_vs_block_diff(df: pd.DataFrame, out_path: str):
469     if df.empty:
470         print("proposer rank 散布図を作成できません: 有効なデータがありません。")
471         return
472
473     if "proposer_priority_rank" not in df.columns:
474         print("proposer_priority_rank がないため、散布図を作成できません。")
475         return
476
477     valid_df = df.dropna(subset=["proposer_priority_rank", "block_diff_sec"])
478
479     if valid_df.empty:
480         print("proposer_priority_rank と block_diff_sec の有効データがありません。")
481         return
482
483     plt.figure(figsize=(10, 7))
484     plt.scatter(
485         valid_df["proposer_priority_rank"],
486         valid_df["block_diff_sec"],
487         s=25,
488         alpha=0.75
489     )
490
491     plt.xlabel("Block Proposer Rank")
492     plt.ylabel("Block Time Difference (seconds)")
493     plt.title("Block Proposer Rank vs Block Time Difference")
494     plt.grid(True)
495     plt.tight_layout()
496     plt.savefig(out_path, dpi=200)
497     plt.close()
498
499     print(f保存しました": {out_path}")
500
501
502 def plot_prev_proposer_rank_vs_block_diff(df: pd.DataFrame, out_path: str):
503     if df.empty:
504         print("previous proposer rank 散布図を作成できません: 有効なデータがありません。")
505         return
506
507     if "prev_proposer_priority_rank" not in df.columns:
508         print("prev_proposer_priority_rank がないため、散布図を作成できません。")
509         return
510
511     valid_df = df.dropna(subset=["prev_proposer_priority_rank", "block_diff_sec"])
512
513     if valid_df.empty:
514         print("prev_proposer_priority_rank と block_diff_sec の有効データがありません。")
515         return
516
517     plt.figure(figsize=(10, 7))

```

```

518     plt.scatter(
519         valid_df["prev_proposer_priority_rank"],
520         valid_df["block_diff_sec"],
521         s=25,
522         alpha=0.75
523     )
524
525     plt.xlabel("Previous Block Proposer Rank")
526     plt.ylabel("Block Time Difference (seconds)")
527     plt.title("Previous Block Proposer Rank vs Block Time Difference")
528     plt.grid(True)
529     plt.tight_layout()
530     plt.savefig(out_path, dpi=200)
531     plt.close()
532
533     print(f保存しました: {out_path}")
534
535
536 def plot_current_proposer_prev_rank_over_height(df: pd.DataFrame, out_path: str):
537     if df.empty:
538         print("current proposer previous rank の時系列図を作成できません: 有効なデータがありません。")
539         return
540
541     if "current_proposer_rank_in_prev_block" not in df.columns:
542         print("current_proposer_rank_in_prev_block がないため、描画できません。")
543         return
544
545     valid_df = df.dropna(subset=[
546         "height",
547         "current_proposer_rank_in_prev_block",
548     ])
549
550     if valid_df.empty:
551         print("current_proposer_rank_in_prev_block の有効データがありません。")
552         return
553
554     plt.figure(figsize=(12, 6))
555     plt.plot(
556         valid_df["height"],
557         valid_df["current_proposer_rank_in_prev_block"],
558         marker="o",
559         markersize=3,
560         linewidth=1
561     )
562
563     plt.xlabel("Height")
564     plt.ylabel("Proposer Rank in Previous Block")
565     plt.title("Proposer Rank in Previous Block over Height")
566     plt.grid(True)
567     plt.tight_layout()
568     plt.savefig(out_path, dpi=200)
569     plt.close()
570
571     print(f保存しました: {out_path}")
572
573

```

```

574 def plot_current_proposer_prev_rank_vs_block_diff(df: pd.DataFrame, out_path: str):
575     if df.empty:
576         print("current proposer previous rank 散布図を作成できません: 有効なデータがありませ
ん。")
577         return
578
579     if "current_proposer_rank_in_prev_block" not in df.columns:
580         print("current_proposer_rank_in_prev_block がないため、散布図を作成できません。")
581         return
582
583     valid_df = df.dropna(subset=[
584         "current_proposer_rank_in_prev_block",
585         "block_diff_sec",
586     ])
587
588     if valid_df.empty:
589         print("current_proposer_rank_in_prev_block と block_diff_sec の有効データがありませ
ん。")
590         return
591
592     plt.figure(figsize=(10, 7))
593     plt.scatter(
594         valid_df["current_proposer_rank_in_prev_block"],
595         valid_df["block_diff_sec"],
596         s=25,
597         alpha=0.75
598     )
599
600     plt.xlabel("Proposer Rank in Previous Block")
601     plt.ylabel("Block Time Difference (seconds)")
602     plt.title("Proposer Rank in Previous Block vs Block Time Difference")
603     plt.grid(True)
604     plt.tight_layout()
605     plt.savefig(out_path, dpi=200)
606     plt.close()
607
608     print(f保存しました: {out_path}")
609
610
611 def calc_correlations(df: pd.DataFrame, x_col: str, y_col: str):
612     valid_df = df.dropna(subset=[x_col, y_col])
613
614     if len(valid_df) < 2:
615         return None, None
616
617     pearson = valid_df[x_col].corr(
618         valid_df[y_col],
619         method="pearson"
620     )
621
622     x_rank = valid_df[x_col].rank()
623     y_rank = valid_df[y_col].rank()
624
625     spearman = x_rank.corr(
626         y_rank,
627         method="pearson"
628     )
629

```



```

630     return pearson, spearman
631
632
633 def print_corr(df: pd.DataFrame, x_col: str, y_col: str, title: str):
634     if x_col not in df.columns or y_col not in df.columns:
635         return
636
637     pearson, spearman = calc_correlations(df, x_col, y_col)
638
639     if pearson is not None and spearman is not None:
640         print(f"\nCorrelation: {title}")
641         print(f"    Pearson : {pearson:.6f}")
642         print(f"    Spearman : {spearman:.6f}")
643
644
645 def print_summary(df: pd.DataFrame, presence_df: pd.DataFrame):
646     print("\n==== Signature Span vs Block Interval Summary =====")
647     print(f"merged rows                : {len(df)}")
648     print(f"unique validators in presence: {presence_df['address'].nunique()}")
649
650     if df.empty:
651         print有効な分析データがありません。("")
652         return
653
654     print("\nValidator signature time span:")
655     print(f"    min   : {df['sig_time_span_sec'].min():.6f} sec")
656     print(f"    max   : {df['sig_time_span_sec'].max():.6f} sec")
657     print(f"    mean  : {df['sig_time_span_sec'].mean():.6f} sec")
658     print(f"    median: {df['sig_time_span_sec'].median():.6f} sec")
659     print(f"    p90   : {df['sig_time_span_sec'].quantile(0.90):.6f} sec")
660     print(f"    p99   : {df['sig_time_span_sec'].quantile(0.99):.6f} sec")
661
662     print("\nBlock time difference:")
663     print(f"    min   : {df['block_diff_sec'].min():.6f} sec")
664     print(f"    max   : {df['block_diff_sec'].max():.6f} sec")
665     print(f"    mean  : {df['block_diff_sec'].mean():.6f} sec")
666     print(f"    median: {df['block_diff_sec'].median():.6f} sec")
667     print(f"    p90   : {df['block_diff_sec'].quantile(0.90):.6f} sec")
668     print(f"    p99   : {df['block_diff_sec'].quantile(0.99):.6f} sec")
669
670     print_corr(
671         df,
672         "sig_time_span_sec",
673         "block_diff_sec",
674         "sig_time_span_sec vs block_diff_sec"
675     )
676
677     print_corr(
678         df,
679         "proposer_priority_rank",
680         "block_diff_sec",
681         "proposer_priority_rank vs block_diff_sec"
682     )
683
684     print_corr(
685         df,
686         "prev_proposer_priority_rank",

```

```

687     "block_diff_sec",
688     "prev_proposer_priority_rank vs block_diff_sec"
689 )
690
691 print_corr(
692     df,
693     "current_proposer_rank_in_prev_block",
694     "block_diff_sec",
695     "current_proposer_rank_in_prev_block vs block_diff_sec"
696 )
697
698 print_corr(
699     df,
700     "current_proposer_priority_in_prev_block",
701     "block_diff_sec",
702     "current_proposer_priority_in_prev_block vs block_diff_sec"
703 )
704
705 if "proposer_priority_rank" in df.columns:
706     valid_rank_df = df.dropna(subset=["proposer_priority_rank"])
707
708     if not valid_rank_df.empty:
709         print("\nBlock proposer rank:")
710         print(f"    min    : {valid_rank_df['proposer_priority_rank'].min():.0f}")
711         print(f"    max    : {valid_rank_df['proposer_priority_rank'].max():.0f}")
712         print(f"    mean   : {valid_rank_df['proposer_priority_rank'].mean():.2f}")
713         print(f"    median : {valid_rank_df['proposer_priority_rank'].median():.2f}")
714
715 if "prev_proposer_priority_rank" in df.columns:
716     valid_prev_rank_df = df.dropna(subset=["prev_proposer_priority_rank"])
717
718     if not valid_prev_rank_df.empty:
719         print("\nPrevious block proposer rank:")
720         print(f"    min    : {valid_prev_rank_df['prev_proposer_priority_rank'].min()
↪ :.0f}")
721         print(f"    max    : {valid_prev_rank_df['prev_proposer_priority_rank'].max()
↪ :.0f}")
722         print(f"    mean   : {valid_prev_rank_df['prev_proposer_priority_rank'].mean()
↪ :.2f}")
723         print(f"    median : {valid_prev_rank_df['prev_proposer_priority_rank'].median
↪ ():.2f}")
724
725 if "current_proposer_rank_in_prev_block" in df.columns:
726     valid_current_prev_rank_df = df.dropna(subset=["
↪ current_proposer_rank_in_prev_block"])
727
728     if not valid_current_prev_rank_df.empty:
729         print("\nCurrent proposer rank in previous block:")
730         print(f"    min    : {valid_current_prev_rank_df['
↪ current_proposer_rank_in_prev_block'].min():.0f}")
731         print(f"    max    : {valid_current_prev_rank_df['
↪ current_proposer_rank_in_prev_block'].max():.0f}")
732         print(f"    mean   : {valid_current_prev_rank_df['
↪ current_proposer_rank_in_prev_block'].mean():.2f}")
733         print(f"    median : {valid_current_prev_rank_df['
↪ current_proposer_rank_in_prev_block'].median():.2f}")
734

```

```

735     if "current_proposer_priority_in_prev_block" in df.columns:
736         valid_prev_priority_df = df.dropna(subset=["
↪ current_proposer_priority_in_prev_block"])
737
738         if not valid_prev_priority_df.empty:
739             print("\nCurrent proposer priority in previous block:")
740             print(f"    min    : {valid_prev_priority_df['
↪ current_proposer_priority_in_prev_block'].min():.0f}")
741             print(f"    max    : {valid_prev_priority_df['
↪ current_proposer_priority_in_prev_block'].max():.0f}")
742             print(f"    mean   : {valid_prev_priority_df['
↪ current_proposer_priority_in_prev_block'].mean():.2f}")
743             print(f"    median : {valid_prev_priority_df['
↪ current_proposer_priority_in_prev_block'].median():.2f}")
744
745     if "validator_count" in df.columns:
746         print("\nValidator count:")
747         print(f"    min    : {df['validator_count'].min():.0f}")
748         print(f"    max    : {df['validator_count'].max():.0f}")
749         print(f"    mean   : {df['validator_count'].mean():.2f}")
750
751     if "total_voting_power" in df.columns:
752         print("\nTotal voting power:")
753         print(f"    min    : {df['total_voting_power'].min():.0f}")
754         print(f"    max    : {df['total_voting_power'].max():.0f}")
755         print(f"    mean   : {df['total_voting_power'].mean():.2f}")
756
757
758 def main():
759     os.makedirs(OUT_DIR, exist_ok=True)
760
761     block_df, presence_df, per_height_df = load_data()
762     proposer_rank_df = build_proposer_rank_df(RAW_BLOCK_JSON_DIR)
763
764     merged_df = build_merged_df(
765         block_df,
766         per_height_df,
767         proposer_rank_df
768     )
769
770     merged_csv_path = os.path.join(OUT_DIR, MERGED_CSV)
771
772     merged_df.to_csv(
773         merged_csv_path,
774         index=False,
775         encoding="utf-8-sig"
776     )
777
778     print(f保存しました: {merged_csv_path})
779
780     simple_path = os.path.join(OUT_DIR, SCATTER_SIMPLE)
781     validator_count_path = os.path.join(OUT_DIR, SCATTER_BY_VALIDATOR_COUNT)
782     total_voting_power_path = os.path.join(OUT_DIR, SCATTER_BY_TOTAL_VOTING_POWER)
783     proposer_rank_path = os.path.join(OUT_DIR, SCATTER_PROPOSER_RANK)
784     prev_proposer_rank_path = os.path.join(OUT_DIR, SCATTER_PREV_PROPOSER_RANK)
785     current_proposer_prev_rank_line_path = os.path.join(
786         OUT_DIR,

```

```

787     CURRENT_PROPOSER_PREV_RANK_LINE
788 )
789 current_proposer_prev_rank_scatter_path = os.path.join(
790     OUT_DIR,
791     CURRENT_PROPOSER_PREV_RANK_SCATTER
792 )
793
794 plot_simple_scatter(
795     merged_df,
796     simple_path
797 )
798
799 plot_scatter_by_validator_count(
800     merged_df,
801     validator_count_path
802 )
803
804 plot_scatter_by_total_voting_power(
805     merged_df,
806     total_voting_power_path
807 )
808
809 plot_proposer_rank_vs_block_diff(
810     merged_df,
811     proposer_rank_path
812 )
813
814 plot_prev_proposer_rank_vs_block_diff(
815     merged_df,
816     prev_proposer_rank_path
817 )
818
819 plot_current_proposer_prev_rank_over_height(
820     merged_df,
821     current_proposer_prev_rank_line_path
822 )
823
824 plot_current_proposer_prev_rank_vs_block_diff(
825     merged_df,
826     current_proposer_prev_rank_scatter_path
827 )
828
829 print_summary(merged_df, presence_df)
830
831 print("\n==== Output Files =====")
832 print(merged_csv_path)
833 print(simple_path)
834 print(validator_count_path)
835 print(total_voting_power_path)
836 print(proposer_rank_path)
837 print(prev_proposer_rank_path)
838 print(current_proposer_prev_rank_line_path)
839 print(current_proposer_prev_rank_scatter_path)
840
841 return 0
842
843

```

```
844 if __name__ == "__main__":
845     raise SystemExit(main())
```

このスクリプトにより、ブロック高ごとのバリデータ数、Voting Power の合計、バリデータ集合の追加・削除、Voting Power や Proposer Priority の変化を分析できる。また、CSV および図として出力されるため、Proposer-Visualizer や表計算ソフトと組み合わせた分析にも利用できる。

Day1: 必須課題 1-4

ソースコード 1-4 を用いて、1-1,1-2,1-3 で取得したデータも用いて分析し、ソースコードの動作を説明し、Block Proposer Rank とブロック生成時間間隔の関係性を確認する散布図、Validator 署名時間とブロック生成時間間隔の関係性を確認する散布図などを描画し、関係性についてについて考察し、報告せよ。

3.6 Proposer-Visualizer

Proposer-Visualizer は、Cosmos / CometBFT 系ブロックチェーンから取得したブロックデータおよびバリデータ関連データを可視化するための Web アプリケーションである。本ツールの目的は、ブロック提案者 (Proposer) の出現傾向、ブロック間隔、バリデータ (Validator) の優先度推移などを視覚的に確認し、チェーン上での挙動を理解しやすくすることである。

このツールでは、取得済みの CSV や JSON を入力として、ブラウザ上で複数の可視化結果を表示する。

共有された実行ファイル群および必要なデータを所定の場所に配置し、指定された手順に従ってアプリケーションを起動することで、同一の可視化結果を確認できる。

起動コマンド例をコマンド 25 に示す。

コマンド 25 Proposer-Visualizer の起動コマンド例

```
1 bash start.sh
```

本実験では、前段で取得したブロックデータおよび分析結果を Proposer-Visualizer に読み込ませ、ブロック提案者の出現傾向、ブロック間隔の特徴、バリデータ情報との関係などを確認する。これにより、ブロックチェーンの合意形成過程を定性的かつ定量的に理解することを目的とする。

練習課題

1-5_visualizer-convert.py を実行し、Proposer-Visualizer 用データを作成した後、Proposer-Visualizer を起動し、分析結果ファイルを読み込ませ Visualizer が起動することを確認せよ。

Day1: 必須課題 1-5

Proposer-Visualizer を起動し、分析結果ファイルを読み込ませたうえで、Validator 遷移および Proposer の出現傾向について考察し、報告せよ。共同実験者の分析結果を読み込ませた結果と比較、考察し、報告せよ。また 1-5 のソースコードの動作を説明すること。

3.7 CW20 トークンの発行

本実験で実装対象とするトークンは、CosmWasm におけるファンジブルトークン標準である **CW20** である。CW20 は、Ethereum における ERC-20 に対応する概念を CosmWasm 向けに定義したものであり、トークンの発行、残高管理、送金、許可 (Allowance) などの基本機能を備える。

CW20 コントラクトでは、一般に初期化時にトークン名、シンボル、decimal 桁数、初期残高、および mint 権限の有無などを設定する。その後、ユーザはコントラクトに対して送金や残高照会などの操作を行うことで、ブロックチェーン上でトークンを扱うことができる。

3.8 Injective の tokenfactory 用いた独自トークンの作成

本節では、Injective の tokenfactory モジュールを用いて、ネイティブ denom 形式の独自トークンを作成する方法を扱う。前節で扱った CW20 はスマートコントラクトとしてトークン状態を管理する方式であるのに対し、Token Factory ではチェーンの標準機能としてトークン denom を作成する。

3.8.1 事前確認

まず、鍵名 mykey のウォレットを作成する。ウォレット作成コマンドをコマンド 26 に示す。

コマンド 26 ウォレット作成コマンド

```
1 injectived keys add mykey
```

次に、testnet 用の共通設定を環境変数として与える。環境変数の設定コマンドをコマンド 27 に示す。

コマンド 27 Injective testnet 用環境変数の設定コマンド

```
1 export CHAIN_ID=injective-888
2 export NODE=https://injective-testnet-rpc.publicnode.com:443
3 export GAS_PRICES=5000000000inj
4 export GAS=1000000
5 export KEY_NAME=testwalet
6 export MY_ADDR=$(injectived keys show $KEY_NAME - a--keyring-backend test)
```

3.8.2 Faucet の実行

<https://cloud.google.com/application/web3/faucet/injective/testnet>

faucet 実行後に、残高が反映されたかを確認する。残高確認コマンドをコマンド 28 に示す。

コマンド 28 faucet 実行後の残高確認コマンド

```
1 injectived query bank balances $MY_ADDR --node=$NODE
```

3.8.3 独自 denom の作成

Token Factory を用いて独自 denom を作成する。独自 denom の作成コマンドをコマンド 29 に示す。

コマンド 29 独自 denom の作成コマンド

```
1 injectived tx tokenfactory create-denom \
```

```

2  "$SUBDENOM" \
3  "My Token" \
4  "MTK" \
5  6 \
6  --from="$KEY_NAME" \
7  --chain-id="$CHAIN_ID" \
8  --node="$NODE" \
9  --gas-prices="$GAS_PRICES" \
10 --gas="$GAS" \
11 -y

```

作成後, denom は次の形式となる.

ソースコード 30 Token Factory で作成される denom の形式例

```

1  factory/${MY_ADDR}/${SUBDENOM}

```

3.8.4 denom metadata の設定

denom metadata の設定コマンドをコマンド 31 に示す.

コマンド 31 denom metadata の設定コマンド

```

1  injected tx tokenfactory set-denom-metadata \
2  "My Token Description" \
3  "$DENOM" \
4  "MTK" \
5  "My Token" \
6  "MTK" \
7  "" \
8  "" \
9  "[{\"denom\":\"$DENOM\",\"exponent\":0,\"aliases\":[]},{\"denom\":\"MTK\",\"exponent
↪ \":6,\"aliases\":[]}]\" \
10 10 \
11 6 \
12 --from="$KEY_NAME" \
13 --chain-id="$CHAIN_ID" \
14 --node="$NODE" \
15 --gas-prices="$GAS_PRICES" \
16 --gas="$GAS" \
17 -y

```

ここでは base denom を \$DENOM, 表示用 denom を MTK とし, $1 \text{ MTK} = 10^6 \text{ base unit}$ となるように設定している.

3.8.5 トークンの mint

mint コマンドをコマンド 32 に示す.

コマンド 32 トークンの mint コマンド

```

1  injected tx tokenfactory mint \
2  "1000000000$DENOM" \
3  "$CREATOR_ADDR" \
4  --from="$KEY_NAME" \
5  --chain-id="$CHAIN_ID" \
6  --node="$NODE" \

```

```

7 --gas-prices="$GAS_PRICES" \
8 --gas="$GAS" \
9 -y

```

3.8.6 残高確認

mint 後に、自アカウントの残高を確認する。残高確認コマンドをコマンド 33 に示す。

コマンド 33 mint 後の残高確認コマンド

```

1 injectived query bank balances $MY_ADDR --node=$NODE

```

3.8.7 INJ の送金確認

独自トークンや CW20 トークンの送金実験に進む前に、Injective testnet 上で通常の INJ 送金を実行し、トランザクションが正しく処理されることを確認する。受信先アドレスを RECIPIENT とすると、送金コマンドはコマンド 34 のようになる。

コマンド 34 INJ の送金確認コマンド

```

1 export RECIPIENT=inj1xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
2
3 injectived tx bank send \
4   $MY_ADDR \
5   $RECIPIENT \
6   10000000000000000inj \
7   --from=$KEY_NAME \
8   --chain-id=$CHAIN_ID \
9   --node=$NODE \
10  --gas-prices=$GAS_PRICES \
11  --gas=$GAS

```

送金後は、トランザクションハッシュをブロックチェーンエクスプローラーで確認し、送金がブロックに取り込まれたことを確認する。

Day1: 必須課題 2-1

INJ を他の共同実験者間で送金し、送金結果をブロックチェーンエクスプローラーで確認し報告せよ。
また、TX Hash と送金完了までに要した時間、10 回以上の平均時間をまとめ報告せよ。

3.8.8 Token Factory による独自トークンの送金

作成した独自トークンがネイティブ資産として扱われることを確認するため、別アカウントへ送金する。

コマンド 35 独自トークンの送金コマンド

```

1 export RECIPIENT=inj1xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
2
3 injectived tx bank send \
4   $MY_ADDR \
5   $RECIPIENT \
6   1000000${DENOM} \
7   --from=$KEY_NAME \

```



```

8  --chain-id=$CHAIN_ID \
9  --node=$NODE \
10 --gas-prices=$GAS_PRICES \
11 --gas=$GAS

```

Day1: 必須課題 2-2

Token factory で作成した独自トークンを他の共同実験者間で送金し、送金結果をブロックチェーンエクスプローラーで確認し報告せよ。また、TX Hash と送金完了までに要した時間、10 回以上の平均時間をまとめ報告せよ。

3.8.9 トークンの burn

作成者 admin は、自身の token を burn することもできる。

コマンド 36 トークンの burn コマンド

```

1 injected tx tokenfactory burn 1000000${DENOM} \
2   --from=$KEY_NAME \
3   --chain-id=$CHAIN_ID \
4   --node=$NODE \
5   --gas-prices=$GAS_PRICES \
6   --gas=$GAS

```

3.9 CW20 コントラクトを用いた独自トークンの作成

前節では、Injective の tokenfactory モジュールを用いた独自トークンの作成方法を扱った。これに対し、本節では CosmWasm のスマートコントラクトとして CW20 を利用し、そのコントラクトを用いて独自トークンを作成する方法を扱う。本実験では、独自に CW20 コントラクトを一から実装するのではなく、CosmWasm の cw-plus リポジトリに含まれる cw20-base をサンプルコードとして利用する [10]。

3.9.1 サンプルコードの取得

コマンド 37 CW20 サンプルコードの取得コマンド

```

1 git clone https://github.com/CosmWasm/cw-plus.git
2 cd cw-plus/contracts/cw20-base

```

3.9.2 サンプルコードのビルド

コマンド 38 CW20 サンプルコードのビルドコマンド

```

1 cargo wasm
2 cp ../../target/wasm32-unknown-unknown/release/cw20_base.wasm .

```

3.9.3 コントラクトコードの保存

コマンド 39 CW20 コントラクト保存コマンド例

```
1 injected tx wasm store cw20_base.wasm \  
2   --from=$KEY_NAME \  
3   --chain-id=$CHAIN_ID \  
4   --node=$NODE \  
5   --gas-prices=$GAS_PRICES \  
6   --gas=3000000
```

3.9.4 初期化メッセージの作成

ソースコード 40 CW20 コントラクトの初期化メッセージ例

```
1 {  
2   "name": "My CW20 Token",  
3   "symbol": "MCW",  
4   "decimals": 6,  
5   "initial_balances": [  
6     {  
7       "address": "inj1xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx",  
8       "amount": "1000000"  
9     }  
10  ],  
11   "mint": {  
12     "minter": "inj1xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx",  
13     "cap": null  
14   },  
15   "marketing": null  
16 }
```

3.9.5 コントラクトのインスタンス化

コマンド 41 CW20 コントラクトのインスタンス化コマンド例

```
1 injected tx wasm instantiate <CODE_ID> cw20_init.json \  
2   --label "my-cw20-token" \  
3   --no-admin \  
4   --from=$KEY_NAME \  
5   --chain-id=$CHAIN_ID \  
6   --node=$NODE \  
7   --gas-prices=$GAS_PRICES \  
8   --gas=2000000
```

3.9.6 残高照会

コマンド 42 CW20 残高照会コマンド例

```
1 injected query wasm contract-state smart <CONTRACT_ADDRESS> \  
2   '{"balance":{"address":"inj1xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx"}}' \  
3   --node=$NODE
```

3.9.7 送金処理

CW20 の送金は、コントラクトに対して `transfer` メッセージを送ることで実行する。

コマンド 43 CW20 の送金コマンド例

```
1 injectived tx wasm execute <CONTRACT_ADDRESS> \  
2   '{"transfer":{"recipient":"inj1yyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyy","amount"  
↪   "":"500000"}}}' \  
3   --from=$KEY_NAME \  
4   --chain-id=$CHAIN_ID \  
5   --node=$NODE \  
6   --gas-prices=$GAS_PRICES \  
7   --gas=300000
```

3.9.8 8. mint および burn

mint 権限を持つアドレスが設定されている場合には、追加発行を行うこともできる。

ソースコード 44 CW20 の mint メッセージ例

```
1 {  
2   "mint": {  
3     "recipient": "inj1xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx",  
4     "amount": "1000000"  
5   }  
6 }
```

Day1: 必須課題 2-3

CW-20 のスマートコントラクトで作成した独自トークンを他の共同実験者間で送金し、送金結果をブロックチェーンエクスプローラーで確認し報告せよ。また、TX Hash と送金完了までに要した時間、10 回以上の平均時間、他の共同実験者から受け取った CW20 トークンが、自身のウォレット上で確認できたことをまとめ報告せよ。

Day1: 必須課題 2-4

INJ, Token Factory による独自トークン、CW20 トークンの送金時間を比較し、それぞれの時間特性について考察せよ。

以下に、今回の CW20 コントラクトの store, instantiate, query, transfer までのコマンド例をまとめる。

3.9.9 CW20 コントラクト操作の環境変数設定

まず、Injective testnet 上で CW20 コントラクトを操作するための環境変数を設定する。環境変数の設定コマンドをコマンド 45 に示す。

コマンド 45 CW20 コントラクト操作の環境変数設定コマンド

```
1 export KEY_NAME="yamalog-nodewallet1"  
2 export CHAIN_ID="injective-888"  
3 export NODE="https://testnet.sentry.tm.injective.network:443"  
4 export GAS_PRICES="500000000inj"
```

鍵情報を確認するコマンドをコマンド 46 に示す。

コマンド 46 鍵情報の確認コマンド

```
1 injectived keys list --keyring-backend test
```

自分のアドレスを確認するコマンドをコマンド 47 に示す。

コマンド 47 自分のアドレス確認コマンド

```
1 injectived keys show "$KEY_NAME" -a --keyring-backend test
```

自分のアドレスを環境変数として保存するコマンドをコマンド 48 に示す。

コマンド 48 自分のアドレスを環境変数に保存するコマンド

```
1 export ADDR=$(injectived keys show "$KEY_NAME" -a --keyring-backend test)
2 echo "$ADDR"
```

3.9.10 WASM コントラクトの保存

次に、ビルド済みの CW20 WASM コントラクトをチェーン上に保存する。保存コマンドをコマンド 49 に示す。

コマンド 49 CW20 WASM コントラクトの保存コマンド

```
1 injectived tx wasm store artifacts/cw20_base.wasm \
2   --from="$KEY_NAME" \
3   --chain-id="$CHAIN_ID" \
4   --node="$NODE" \
5   --gas-prices="$GAS_PRICES" \
6   --gas=3000000 \
7   --keyring-backend test \
8   --broadcast-mode sync \
9   -y
```

トランザクションハッシュが返された後、その結果を確認する。確認コマンドをコマンド 50 に示す。

コマンド 50 store トランザクション結果の確認コマンド

```
1 injectived query tx <TXHASH> \
2   --node="$NODE" \
3   -o json | jq
```

保存されたコントラクトの code_id を取得する。code_id の取得コマンドをコマンド 51 に示す。

コマンド 51 code ID の取得コマンド

```
1 injectived query tx <TXHASH> \
2   --node="$NODE" \
3   -o json | jq -r '
4     .events[]
5     | select(.type=="store_code")
6     | .attributes[]
7     | select(.key=="code_id")
8     | .value
9   '
```

取得した code_id を環境変数として保存する。例をコマンド 52 に示す。

コマンド 52 code ID の環境変数設定コマンド

```
1 export CODE_ID="39402"
```

3.9.11 instantiate 用メッセージの作成

次に、CW20 コントラクトをインスタンス化するための初期化メッセージを作成する。ここでは、トークン名を Yamalog Token, シンボルを YMLG, 小数桁数を 6 とする。初期化メッセージの作成コマンドをコマンド 53 に示す。

コマンド 53 CW20 初期化メッセージの作成コマンド

```
1 export ADDR=$(injectived keys show "$KEY_NAME" -a --keyring-backend test)
2
3 INIT='{
4   "name": "Yamalog Token",
5   "symbol": "YMLG",
6   "decimals": 6,
7   "initial_balances": [
8     {
9       "address": "'"$ADDR"'",
10      "amount": "1000000000"
11    }
12  ],
13   "mint": {
14     "minter": "'"$ADDR"'",
15     "cap": "1000000000000"
16   },
17   "marketing": null
18 }'
```

作成した初期化メッセージを確認するコマンドをコマンド 54 に示す。

コマンド 54 CW20 初期化メッセージの確認コマンド

```
1 echo "$INIT" | jq
```

3.9.12 コントラクトのインスタンス化

保存した code_id と初期化メッセージを用いて、CW20 コントラクトをインスタンス化する。インスタンス化コマンドをコマンド 55 に示す。

コマンド 55 CW20 コントラクトのインスタンス化コマンド

```
1 injectived tx wasm instantiate "$CODE_ID" "$INIT" \
2   --from="$KEY_NAME" \
3   --label="cw20-base-yamalog-token" \
4   --admin="$ADDR" \
5   --chain-id="$CHAIN_ID" \
6   --node="$NODE" \
7   --gas-prices="$GAS_PRICES" \
8   --gas=1000000 \
9   --keyring-backend test \
10  --broadcast-mode sync \
11  -y
```

トランザクションハッシュが返された後、その結果を確認する。確認コマンドをコマンド 56 に示す。

コマンド 56 instantiate トランザクション結果の確認コマンド

```
1 injected query tx <TXHASH> \  
2 --node="$NODE" \  
3 -o json | jq
```

インスタンス化された CW20 コントラクトアドレスを取得する。取得コマンドをコマンド 57 に示す。

コマンド 57 CW20 コントラクトアドレスの取得コマンド

```
1 injected query tx <TXHASH> \  
2 --node="$NODE" \  
3 -o json | jq -r '  
4   .events[  
5     | select(.type=="instantiate" or .type=="instantiate_contract")  
6     | .attributes[  
7     | select(.key=="_contract_address" or .key=="contract_address")  
8     | .value  
9   ]
```

取得したコントラクトアドレスを環境変数として保存する。例をコマンド 58 に示す。

コマンド 58 CW20 コントラクトアドレスの環境変数設定コマンド

```
1 export CONTRACT_ADDR="inj16aljyql0qjedjpnd3zdga5q5gpwaheautvwnp8"
```

3.9.13 CW20 コントラクト情報の確認

インスタンス化後、CW20 コントラクトの基本情報を確認する。トークン情報の確認コマンドをコマンド 59 に示す。

コマンド 59 CW20 token.info の確認コマンド

```
1 injected query wasm contract-state smart "$CONTRACT_ADDR" \  
2 '{"token_info":{}}' \  
3 --node="$NODE" \  
4 -o json | jq
```

自分の CW20 残高を確認するコマンドをコマンド 60 に示す。

コマンド 60 自分の CW20 残高確認コマンド

```
1 injected query wasm contract-state smart "$CONTRACT_ADDR" \  
2 '{"balance":{"address":"'"$ADDR"'"}}' \  
3 --node="$NODE" \  
4 -o json | jq
```

minter 情報を確認するコマンドをコマンド 61 に示す。

コマンド 61 CW20 minter 情報の確認コマンド

```
1 injected query wasm contract-state smart "$CONTRACT_ADDR" \  
2 '{"minter":{}}' \  
3 --node="$NODE" \  
4 -o json | jq
```

3.9.14 CW20 トークン送金

CW20 トークンを他のアドレスへ送金するため、送金先アドレスと送金量を設定する。送金先アドレスの設定コマンドをコマンド 62 に示す。

コマンド 62 CW20 送金先アドレスの設定コマンド

```
1 export RECIPIENT_ADDR="inj1xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx"
```

送金量の設定コマンドをコマンド 63 に示す.

コマンド 63 CW20 送金量の設定コマンド

```
1 export SEND_AMOUNT="1000000"
```

本コントラクトでは decimals が 6 であるため、1000000 は 1 YMLG に相当する。

CW20 トークンの送金実行コマンドをコマンド 64 に示す。

コマンド 64 CW20 トークン送金コマンド

```
1 injected tx wasm execute "$CONTRACT_ADDR" \
2   '{"transfer":{"recipient":"'"$RECIPIENT_ADDR"'',"amount":"'"$SEND_AMOUNT"'"}}' \
3   --from="$KEY_NAME" \
4   --chain-id="$CHAIN_ID" \
5   --node="$NODE" \
6   --gas-prices="$GAS_PRICES" \
7   --gas=300000 \
8   --keyring-backend test \
9   --broadcast-mode sync \
10  -v
```

送金トランザクションの結果を確認するコマンドをコマンド 65 に示す.

コマンド 65 CW20 送金トランザクション結果の確認コマンド

```
1 injected query tx <TXHASH> \
2 --node="$NODE" \
3 -o json | jq
```

3.9.15 送金後の残高確認

送金後に、自分の残高と送金先の残高を確認する。自分の残高確認コマンドをコマンド 66 に示す。

コマンド 66 CW20 送金後の自分の残高確認コマンド

```
1 injected query wasm contract-state smart "$CONTRACT_ADDR" \
2   '{"balance":{"address":"","$ADDR"},"'} \
3   --node="$NODE" \
4   -o json | jq
```

送金先の残高確認コマンドをコマンド 67 に示す.

コマンド 67 CW20 送金後の送金先残高確認コマンド

```
1 injected query wasm contract-state smart "$CONTRACT_ADDR" \
2   '{"balance":{"address":"","$RECIPIENT_ADDR","}}', \
3   --node="$NODE" \
4   -o json | jq
```

表 1 待ち行列モデルとブロックチェーン処理の対応関係

待ち行列モデル	ブロックチェーンにおける対応
客	トランザクション
到着率 λ	単位時間あたりの Tx 発生率
サービス率 μ	単位時間あたりの Tx 処理能力
待ち行列	mempool
待ち時間	Tx がブロックに取り込まれるまでの時間
サービス完了	Tx がブロックに格納されること

4 Day2: ブロックチェーンの局所シミュレーション

4.1 待ち行列シミュレーション (2-1)

4.1.1 目的

本課題では、Python3 を用いて待ち行列シミュレーションを実行し、到着率、サービス率、利用率、平均待ち時間の関係を確認する。また、この待ち行列モデルをブロックチェーンにおけるトランザクション処理に対応させ、トランザクション発生量がブロック処理能力に近づいた場合、または上回った場合に、待ち時間がどのように変化するかを観察する。

本課題を通して、以下の内容を理解することを目的とする。

- 到着率 λ とサービス率 μ の意味
- 利用率 $\rho = \lambda/\mu$ の意味
- 利用率が 1 に近づくと待ち時間が増加すること
- 到着率がサービス率を上回ると待ち行列が不安定になること
- ブロックチェーンの mempool 混雑と待ち行列モデルの関係

待ち行列モデルでは、利用者や要求がシステムに到着し、サーバによって処理される状況を表現する。本課題では、待ち行列モデルとブロックチェーンにおけるトランザクション処理を、表 1 のように対応させる。

ブロックチェーンでは、トランザクションが短時間に大量発生すると、すべての Tx をすぐにブロックへ格納できない場合がある。このとき、未処理 Tx は mempool に残り、ブロックに取り込まれるまで待機する。

Day2: 必須課題 1

表 2 に示す待ち行列シミュレーションの実行シナリオをすべて実行し、結果を考察せよ。各シナリオについて、平均待ち時間、平均滞在時間、最大待ち時間を確認し、利用率 $\rho = \lambda/\mu$ と待ち時間の関係を説明せよ。特に、 ρ が 1 に近づく場合と、 $\rho > 1$ となる場合に、待ち時間がどのように変化するかを考察せよ。

表2 待ち行列シミュレーションの実行シナリオ

到着率 λ	サービス率 μ	利用率 ρ	意味
0.10	1.00	0.10	非常に空いている状態
0.30	1.00	0.30	空いている状態
0.50	1.00	0.50	中程度の負荷
0.70	1.00	0.70	通常負荷
0.85	1.00	0.85	混雑状態
0.95	1.00	0.95	高混雑状態
0.99	1.00	0.99	処理限界に近い状態
1.10	1.00	1.10	到着率が処理能力を超過している状態

4.1.2 手数料市場シミュレーション (2-2)

Day2: 必須課題 2

「valid.modes = 」が変化したときの Tx ごとの待ちブロック数、fee と待ちブロック数の関係、Tx サイズと待ちブロック数の関係、手数料効率と待ちブロック数の関係、ブロックごとの mempool サイズ推移を観測し、考察せよ。

4.1.3 バリデータ選出シミュレーション (2-3)

PoS (Proof of Stake) 型のブロックチェーンでは、ブロック生成や合意形成に参加するノードを validator と呼ぶ。validator は、保有または委任された stake 量に応じて、ネットワーク上での投票力を持つ。この投票力は一般に voting power と呼ばれ、ブロック提案者の選出や合意形成における重みとして用いられる。

PoS では、voting power が大きい validator ほど、ブロック提案者、すなわち proposer に選ばれる機会が多くなる傾向がある。これは、stake を多く持つ validator ほどネットワークに対する経済的利害が大きいと考えられるためである。一方で、voting power が少数の validator に集中すると、ブロック生成や合意形成における影響力も一部の validator に偏る可能性がある。そのため、PoS ネットワークでは、voting power の分布や上位 validator への集中度を確認することが重要である。

本シミュレーションでは、validator ごとに voting power を設定し、その voting power に比例した確率で proposer を選出する。これにより、voting power が大きい validator ほど proposer に選ばれやすいことを確認する。また、proposer に選ばれた回数を集計することで、voting power の偏りがブロック提案機会の偏りにつながるかを観察する。

本教材では、voting power の生成方法として、均等分布、ランダム分布、上位集中型分布、パレート分布を扱う。均等分布ではすべての validator が同じ voting power を持つため、proposer 選出回数もおおむね均等になることが期待される。一方、パレート分布では、少数の validator が大きな voting power を持ち、多数の validator が小さな voting power を持つ。このため、パレート分布は、現実の PoS ネットワークに見られる stake 集中の傾向を簡易的に表現するために有用である。

voting power の集中度を評価する指標として、Nakamoto Coefficient がある。本教材では、BFT 系 PoS を想定し、全 voting power の 3 分の 1 を超えるために必要な最小 validator 数を Nakamoto Coefficient とし

て扱う。この値が小さい場合、少数の validator に voting power が集中していることを意味する。一方、この値が大きい場合、voting power がより多くの validator に分散していることを意味する。

さらに、本シミュレーションでは、同じ validator が連続して proposer に選ばれる回数も集計する。連続提案は、voting power の大きい validator が短い期間に複数回ブロック提案を行う状況を確認するための指標である。ただし、本シミュレーションは voting power に比例した単純なランダム選出モデルであり、実際の CometBFT における proposer priority アルゴリズムを厳密に再現するものではない。そのため、本シミュレーションは PoS における voting power の偏りと proposer 選出頻度の関係を理解するための教材用モデルとして位置づける。

Day2: 必須課題 3

バリデータ選出シミュレーションを実行し、出力された CSV ファイルおよびグラフを確認せよ。equal, random, top.heavy, pareto の mode の違いや oting power の集中が、proposer 回数の集中にもつながっているかなどの観点について考察せよ。

4.2 IBC の通信シミュレーション (2-4)

IBC (Inter-Blockchain Communication) は、Cosmos エコシステムにおいて異なるブロックチェーン間でデータやトークンを転送するための通信プロトコルである。IBC を用いることで、あるチェーンで発行されたトークンやパケットを、別のチェーンに安全に伝達できる。この仕組みにより、複数の独立したブロックチェーンが相互に接続され、クロスチェーンアプリケーションを構成できる。

IBC 転送では、単に送信元チェーンから送信先チェーンへ直接データが送られるわけではない。一般に、送信元チェーンでパケットが生成され、その情報を relayer が検知し、送信先チェーンへ中継する。その後、送信先チェーンで受信処理が行われ、必要に応じて acknowledgement が送信元チェーンへ返される。したがって、IBC 転送時間は、複数の処理段階に分解して考える必要がある。

本教材では、IBC 転送遅延を単純化し、次の 3 つの要素に分けて扱う。

1. 送信元チェーンで送信 Tx がブロックに取り込まれるまでの待ち時間
2. relayer がパケットを検知し、送信先チェーンへ中継するまでの遅延
3. 送信先チェーンで受信 Tx がブロックに取り込まれるまでの待ち時間

このとき、合計 IBC 転送時間は次のように表せる。

$$T_{IBC} = T_{ChainA} + T_{Relayer} + T_{ChainB}$$

ここで、 T_{ChainA} は送信元チェーン側の待ち時間、 $T_{Relayer}$ は relayer による中継遅延、 T_{ChainB} は送信先チェーン側の待ち時間を表す。送信元チェーンまたは送信先チェーンのブロック生成間隔が長い場合、Tx がブロックに取り込まれるまでの待ち時間が長くなり、IBC 転送全体の遅延も増加する。また、relayer の監視間隔や処理性能、ネットワーク状態によっても転送時間は変化する。

本シミュレーションでは、これらの遅延要因を厳密に再現するのではなく、平均ブロック時間や平均 relayer 遅延をもとにランダムなばらつきを与えることで、IBC 転送時間の基本的な傾向を確認する。具体的には、Chain A の待ち時間、relayer 遅延、Chain B の待ち時間をそれぞれ生成し、それらを合計することで 1 回の

IBC 転送時間を求める。この処理を複数回繰り返すことで、転送ごとのばらつき、平均遅延、遅延要因の内訳を観察できる。

このモデルは実際の IBC 転送を完全に再現するものではないが、IBC 転送においてどの処理段階が遅延に影響するかを理解するための基礎として有用である。特に、送信元チェーンと送信先チェーンのブロック時間差、relay 遅延、および各要素のばらつきが、合計転送時間にどのように影響するかを確認できる。

Day2: 必須課題 4

IBC 転送遅延を送信元チェーンの確定待ち時間、relay の中継遅延、送信先チェーンの確定待ち時間の 3 要素に分解して評価ができる。10000 回の IBC 転送を模擬し、その遅延について考察せよ。

4.3 PoS proposer 偏りシミュレーション (2-5)

Day2: 必須課題 5

4.4 ステーキング報酬シミュレーション (2-6)

Day2: 必須課題 6

4.5 学習用楕円曲線暗号 (ECC) による暗号化と ECDSA 署名 (2-7)

4.5.1 学習目標

本資料では、楕円曲線暗号 (ECC) を用いて、次の内容を理解することを目的とする。

1. 有限体上の楕円曲線の基本
2. 公開鍵・秘密鍵の生成
3. 受信者公開鍵を用いた暗号化
4. 送信者秘密鍵を用いた電子署名
5. 受信者による署名検証と復号
6. 改ざん検出の仕組み

4.5.2 本資料で扱う安全性の観点

本教材では、メッセージ通信において重要な次の 3 つの性質を扱う。

- **秘匿性:** 第三者が本文を読めないこと
- **真正性:** 送信者本人が送信したことを確認できること
- **完全性:** 途中で改ざんされていないこと

4.5.3 教材の位置づけ

本教材は学習用であり、実運用を目的としない。そのため、計算の追跡や可視化を容易にする目的で、小さい有限体上の楕円曲線を使用する。

実運用では、Bitcoin や Ethereum の EOA 署名などにおいて `secp256k1` が用いられる。一方、本教材では、暗号処理の流れを理解しやすくすることを優先する。

4.5.4 使用する教材プログラム

本教材では、次の2つのファイルを用いる。

- `sender_secure.py`
- `receiver_secure.py`

■`sender_secure.py` の役割 送信者側プログラムでは、次の処理を行う。

- 送信者鍵の生成
- 受信者公開鍵を用いた暗号化
- 送信者秘密鍵を用いた署名
- 送信データ JSON の生成

■`receiver_secure.py` の役割 受信者側プログラムでは、次の処理を行う。

- 受信者鍵の生成
- 送信者署名の検証
- 暗号文の復号
- 平文メッセージの出力

Day2: 必須課題 7

`sender_secure.py` と `receiver_secure.py` を実行し、ECC による暗号化、ECDSA 署名、署名検証、復号の流れを確認せよ。また、送信データの一部を変更した場合に、署名検証または復号結果がどのように変化するかを確認し、改ざん検出の仕組みについて考察せよ。

4.6 楕円曲線の計算 (2-8)

本課題では、小さい有限体上の楕円曲線を用いて、楕円曲線上の点の計算を確認する。有限体上の楕円曲線は、次式で表される。

$$y^2 \equiv x^3 + ax + b \pmod{p}$$

ここで、 p は素数であり、 a および b は曲線のパラメータである。本教材では、学習しやすい小さな値を用いて、楕円曲線上の点の分布や演算の性質を確認する。

表 3 PoW 模擬プログラムで用いる主な値

変数	意味
<code>previous_block_hash</code>	直前ブロックのハッシュ
<code>merkle_root</code>	ブロック内 Tx をまとめた Merkle root の模擬値
<code>timestamp</code>	現在時刻
<code>nonce</code>	条件を満たすまで変化させる値

Day2: 必須課題 8

楕円曲線の可視化プログラムを実行し、実数上の楕円曲線と有限体上の点の違いを確認せよ。また、曲線パラメータ a , b , および素数 p を変更し、曲線の形状や有限体上の点の配置がどのように変化するかを考察せよ。

4.7 ハッシュレート (2-9)

このソースコードは、Bitcoin の Proof of Work における nonce 探索を簡易的に模擬するプログラムである。実際の Bitcoin ブロックヘッダを完全に再現しているわけではなく、学習用に、次の値を連結したものを疑似ブロックヘッダとして扱う。

```
previous_block_hash + merkle_root + timestamp + nonce
```

この疑似ブロックヘッダに対して SHA-256 を 2 回計算し、指定した難易度条件を満たす nonce を探索する。

4.7.1 全体の流れ

このプログラムの処理の流れは次のとおりである。

1. 前ブロックハッシュ, Merkle root, timestamp を用意する。
2. `difficulty_zeros` から難易度条件を作成する。
3. nonce を 0 から順に増やす。
4. 疑似ブロックヘッダを作成する。
5. SHA-256 を 2 回計算する。
6. ハッシュ値の先頭が指定個数の 0 で始まるか確認する。
7. 条件を満たせば成功として終了する。
8. 結果を CSV に保存する。

4.7.2 重要な設定

ブロックヘッダの材料となる主な値を表 3 に示す。

実際の Bitcoin では、ブロックヘッダには version, previous block hash, Merkle root, time, bits, nonce などが含まれる。本教材のプログラムでは、PoW の基本原理を理解しやすくするため、これらを簡略化している。

表 4 difficulty_zeros と探索条件の対応

difficulty_zeros	条件
3	ハッシュが 000 で始まる
4	ハッシュが 0000 で始まる
5	ハッシュが 00000 で始まる

表 5 difficulty_zeros と平均試行回数の目安

difficulty_zeros	平均試行回数の目安
3	約 4,096 回
4	約 65,536 回
5	約 1,048,576 回
6	約 16,777,216 回

4.7.3 difficulty の設定

本プログラムでは、ハッシュ値の先頭に必要な 0 の個数を `difficulty_zeros` として指定する。たとえば、`difficulty_zeros = 4` の場合、次の条件を満たす hash を探索する。

```
hash.startswith("0000")
```

難易度条件の例を表 4 に示す。

4.7.4 nonce 探索

中心となる処理は、nonce を 0 から順に変化させ、条件を満たす hash を探索する部分である。疑似ブロックヘッダは、前ブロックハッシュ、Merkle root、timestamp、nonce を連結して作成する。その後、SHA-256 を 2 回計算する。これは Bitcoin のブロックハッシュで用いられる double SHA-256 を模擬したものである。

4.7.5 処理時間とハッシュレート

プログラムでは、`time.perf_counter()` を用いて nonce 探索にかかった時間を計測する。さらに、試行回数を経過時間で割ることで、1 秒あたりのハッシュ試行回数を求める。

$$\text{hash rate} = \frac{\text{checked count}}{\text{elapsed time}}$$

単位は hashes/second である。

4.7.6 難易度を変えた場合の目安

16 進数のハッシュで先頭 0 を 1 つ増やすと、平均試行回数は約 16 倍になる。難易度ごとの平均試行回数の目安を表 5 に示す。

授業や実験では、まず `difficulty_zeros = 4` で動作確認し、処理時間の差を確認したい場合は `difficulty_zeros = 5` に変更するとよい。

表 6 本教材プログラムと実際の Bitcoin の違い

項目	本教材プログラム	実際の Bitcoin
ブロックヘッダ	文字列連結	80 バイトのバイナリ構造
難易度	先頭 0 の個数	target 値との大小比較
nonce	Python 整数を文字列化	32bit nonce
Merkle root	固定文字列	実 Tx から計算
ハッシュ	double SHA-256	double SHA-256

4.7.7 実際の Bitcoin との違い

本プログラムは Bitcoin の考え方を理解するための簡易版である。実際の Bitcoin との差異を表 6 に示す。ただし、nonce を変えながら条件を満たす hash を探索するという PoW の基本理解には十分利用できる。

Day2: 必須課題 9

PoW 模擬プログラムを実行し、difficulty_zeros の値を変更したときに、nonce 探索にかかる時間とハッシュレートがどのように変化するかを確認せよ。少なくとも difficulty_zeros = 4 と difficulty_zeros = 5 の結果を比較し、難易度が高くなるほど試行回数と処理時間が増加する理由を考察せよ。

4.8 PoW ブロックチェーンのマイニング、改ざんの検出、改ざんの完遂 (2-10-1, 2-10-2, 2-10-3)

本課題では、PoW を用いた簡易ブロックチェーンを対象として、ブロック生成、改ざん検出、および改ざん点以降の再計算を確認する。

Day2: 必須課題 10

2-10-1 を用いて 1000 ブロックをマイニングし、2-10-2 を用いて配布ブロックデータから改ざん箇所を検出せよ。さらに、2-10-3 を用いて改ざん点以降のブロックを再計算し、改ざんを完遂するために必要な処理について考察せよ。特に、PoW を用いる場合、改ざん点以降の hash だけでなく nonce も再探索する必要があることを説明せよ。

4.9 Nonce の計算 (2-11)

本課題では、指定したメッセージに対して nonce を変化させながらハッシュ値を計算し、条件を満たす nonce を探索する。これにより、PoW における nonce 探索の基本的な考え方を確認する。

Day2: 必須課題 11

MESSAGE = "Hello, blockchain!" の部分を自身の学籍番号に変更し, nonce の計算を実行せよ。
得られた nonce, hash, 試行回数, 処理時間を記録し, 難易度条件を変更した場合に結果がどのように変化するかを考察せよ。

4.10 総当たり探索の時間見積り (2-12)

探索条件が厳しくなるほど, 必要な試行回数が急激に増加することを確認する。

Day2: 必須課題 12

総当たり探索において, 探索空間の大きさ, 1 秒あたりの試行回数, および推定処理時間の関係を計算せよ。

5 Day3

編集中

参考文献

- [1] Jae Kwon and Ethan Buchman, *Cosmos: A Network of Distributed Ledgers*, Cosmos Whitepaper. Available: <https://github.com/cosmos/cosmos/raw/master/whitepaper/cosmos-sdk-intro.pdf> Accessed: 2026-04-21.
- [2] Cosmos Labs, "What is the Cosmos SDK," Cosmos SDK Documentation. Available: <https://docs.cosmos.network/sdk/v0.53/learn/intro/overview> Accessed: 2026-04-21.
- [3] Cosmos Labs, "IBC-Go Documentation: Introduction," Cosmos Documentation. Available: <https://docs.cosmos.network/ibc/latest/intro> Accessed: 2026-04-21.
- [4] Cosmos IBC Team, "Inter-Blockchain Communication Protocol (IBC)," cosmos/ibc repository. Available: <https://github.com/cosmos/ibc> Accessed: 2026-04-21.
- [5] CometBFT Team, "CometBFT RPC Specification," CometBFT Documentation. Available: <https://docs.cometbft.com/main/spec/rpc/> Accessed: 2026-04-21.
- [6] CometBFT Team, "Validators," CometBFT Documentation. Available: <https://docs.cometbft.com/main/core/validators> Accessed: 2026-04-21.
- [7] Injective Labs, "Creating Tokens," Injective Docs. Available: <https://docs.injective.network/developers/assets/token-create> Accessed: 2026-04-21.
- [8] Injective Labs, "Launch a Token," Injective Docs. Available: <https://docs.injective.network/developers-defi/token-launch> Accessed: 2026-04-21.
- [9] Injective Labs, "Commands," Injective Docs. Available: <https://docs.injective.network/developers/injectived/advanced> Accessed: 2026-04-21.

- [10] CosmWasm Team, “cw20-base,” in *cw-plus*. Available: <https://github.com/CosmWasm/cw-plus/tree/main/contracts/cw20-base> Accessed: 2026-04-21.
- [11] Injective Labs, “Your First CosmWasm Smart Contract,” Injective Docs. Available: <https://docs.injective.network/developers-cosmwasm/smart-contracts/your-first-smart-contract> Accessed: 2026-04-21.
- [12] FUJITA TAKUYA, “Cosmo SDK ベースのチェーン『Neutron』でのトークン発行をしてみよう！”, STIR LAB. Available: <https://lab.stir.network/neutron-token-mint/> Accessed: 2026-04-21.
- [13] SimBlock Project, “SimBlock,” Available: <https://github.com/dsg-titech/simblock> Accessed: 2026-04-21.
- [14] SimBlock Project, “SimBlock Visualizer,” Available: <https://dsg-titech.github.io/simblock-visualizer/> Accessed: 2026-04-21.
- [15] Wikipedia, “Wide Area Network,” Available: https://ja.wikipedia.org/wiki/Wide_Area_Network Accessed: 2026-04-21.

付録 A 初学者のための用語集

本付録では、本実験で扱う主要な用語を整理する。ブロックチェーン (Blockchain)、Cosmos、CometBFT、Injective、CW20、SimBlock などの内容を理解するためには、各用語の意味をあらかじめ把握しておくことが重要である。以下の用語は実験中に繰り返し登場するため、必要に応じて参照すること。

A.1 ブロックチェーンに関する基本用語

ブロックチェーン (Blockchain) 取引データや状態更新の情報をブロック単位で記録し、それらを時系列に連結して管理する分散型台帳である。一度記録されたデータを改ざんするには、後続のブロックも含めて変更する必要があるため、改ざん耐性を持つ構造として利用される。

ブロック (Block) ブロックチェーンにおける記録単位である。ブロックには、ブロック高、生成時刻、提案者、トランザクション、署名情報などが含まれる。

ブロック高 (Block Height) ブロックチェーン上で各ブロックに付与される番号である。一般に、チェーンの先頭から何番目のブロックであるかを表す。たとえば、ブロック高 1000 は、そのチェーンにおける 1000 番目のブロックを意味する。

トランザクション (Transaction) ブロックチェーン上で実行される操作や取引のことである。送金、スマートコントラクト実行、トークン発行、投票などがトランザクションとして記録される。

アドレス (Address) ブロックチェーン上のアカウントを識別するための文字列である。送金元、送金先、コントラクト管理者、バリデータなどを識別するために用いられる。

残高 (Balance) あるアドレスが保有しているトークン量である。本実験では、`injectived query bank balances` などのコマンドにより確認する。

手数料 (Fee) トランザクションをブロックチェーンに送信する際に支払う費用である。手数料は、トランザクション処理やネットワーク利用に対する対価として扱われる。

ファイナリティ (Finality) あるトランザクションやブロックが確定し、原則として取り消されない状態になる性質である。ファイナリティが短いほど、利用者は早く取引確定を確認できる。

スマートコントラクト (Smart Contract) ブロックチェーン上で実行されるプログラムである。あらかじめ定義された条件や処理に従って、トークン発行、送金、状態管理などを自動的に実行する。

A.2 Cosmos / CometBFT に関する用語

Cosmos 複数の独立したブロックチェーンを相互接続することを目的としたブロックチェーンエコシステムである。「ブロックチェーンのインターネット」という考え方にに基づき、目的ごとに設計されたチェーン同士を連携させることを重視している。

Cosmos SDK Cosmos 系ブロックチェーンを構築するための開発フレームワークである。送金、アカウント管理、ステーキング、ガバナンスなどの機能をモジュールとして利用できる。

CometBFT Cosmos 系チェーンで用いられる BFT 系コンセンサスエンジンである。旧 Tendermint Core の後継として扱われ、ブロック生成や合意形成を担う。

Tendermint BFT Byzantine Fault Tolerance を前提としたコンセンサス方式である。一部のノードが故障

または不正な挙動をしても、ネットワーク全体として正しい合意を得ることを目的とする。

IBC (Inter-Blockchain Communication) 異なるブロックチェーン間でトークンやメッセージをやり取りするための通信プロトコルである。Cosmos における相互運用性を支える中核技術である。

ブロックチェーンのインターネット (Internet of Blockchains) 複数の独立したブロックチェーンを相互接続し、全体として一つの大きなネットワークを構成するという考え方である。インターネットが複数のネットワークを接続して成り立つのと同様に、Cosmos では IBC により複数チェーンを接続する。

RPC ノード (RPC Node) Remote Procedure Call node の略であり、外部プログラムからブロックチェーンの情報を問い合わせたり、トランザクションを送信したりするための接続先である。本実験では、ブロック情報やバリデータ情報の取得に RPC エンドポイントを利用する。

RPC エンドポイント (RPC Endpoint) RPC ノードへアクセスするための URL である。たとえば、`/block`、`/validators`、`/status` などのエンドポイントを通じて、ブロック情報やバリデータ情報を取得する。

チェーン ID (Chain ID) ブロックチェーンネットワークを識別するための ID である。トランザクション送信時には、誤ったネットワークへ送信しないようにチェーン ID を指定する。

バリデータ (Validator) ブロックチェーンの合意形成に参加するノードである。Cosmos / CometBFT 系チェーンでは、バリデータがブロック提案や署名を行い、チェーンの安全性と継続性を支える。

バリデータ集合 (Validator Set) あるブロック高において合意形成に参加するバリデータの集合である。本実験では、各ブロック高における Validator Set を取得し、バリデータ数、投票力 (Voting Power)、提案者優先度 (Proposer Priority) などを分析する。

投票力 (Voting Power) バリデータが合意形成において持つ投票力である。Voting Power が大きいほど、合意形成における影響力も大きくなる。

提案者 (Proposer) あるブロックを提案するバリデータである。各ブロックでは、バリデータ集合の中から提案者が選ばれ、その提案者がブロックを生成する。

提案者アドレス (Proposer Address) あるブロックを提案したバリデータのアドレスである。ブロックヘッダに含まれ、ブロック生成者を識別するために用いる。

提案者優先度 (Proposer Priority) CometBFT 系の提案者選出に関係する優先度である。本実験では、前ブロックの Proposer Priority と次ブロックの Proposer の関係を分析対象とする。

署名 (Signature) バリデータがブロックに対する合意を示すために付与する暗号学的な証明である。本実験では、ブロック内の署名数を確認することで、合意形成に関わった情報を分析する。

A.3 Injective / CosmWasm / CW20 に関する用語

Injective Cosmos SDK を基盤とするブロックチェーンの一つである。本実験では、Injective testnet を用いて CLI 操作、トークン作成、スマートコントラクト操作を確認する。

injectived Injective チェーンを操作するための CLI 兼ノードデーモンである。鍵管理、残高確認、トランザクション送信、CosmWasm コントラクト操作などに用いる。

Testnet 実験や開発のために用意されたテスト用ネットワークである。本実験では、Injective の testnet である `injective-888` を利用する。

Faucet Testnet 用のトークンを取得するための仕組みである。実験では、Faucet により取得した testnet 用

INJ を用いて手数料支払いやトランザクション実行を行う。

Token Factory Cosmos SDK 系チェーンにおいて、ネイティブ denom を作成するための機能である。

Injective では、`tokenfactory` モジュールを用いて独自トークンを作成できる。

denom トークンの単位や識別名を表す文字列である。Token Factory で作成される denom は、`factory/{creator address}/{subdenom}` の形式をとる。

subdenom Token Factory で作成する独自トークンの名前部分である。作成者アドレスと組み合わせること
で、最終的な denom が決定される。

ネイティブトークン (Native Token) チェーンの標準機能として扱われるトークンである。Token Factory
によって作成された denom は、銀行モジュールなどを通じてネイティブ資産として扱われる。

CosmWasm Cosmos SDK 系チェーン上でスマートコントラクトを実行するための仕組みである。Rust な
どで記述したコントラクトを WASM 形式にビルドし、チェーン上に保存・インスタンス化して利用
する。

WASM (WebAssembly) WebAssembly の略であり、スマートコントラクトを実行可能な形式にコンパ
イルしたものである。CosmWasm では、WASM ファイルをチェーンに保存してコントラクトとして利
用する。

CW20 CosmWasm におけるファンジブルトークン標準である。Ethereum の ERC-20 に相当する概念であ
り、トークンの送金、残高確認、mint, burn, allowance 管理などを扱う。

cw20-base CosmWasm の `cw-plus` リポジトリに含まれる CW20 の基本実装である。本実験では、CW20
コントラクトのサンプルコードとして利用する。

コード ID (Code ID) チェーン上に保存された WASM コントラクトコードを識別する番号である。コン
トラクトをインスタンス化する際に指定する。

コントラクトアドレス (Contract Address) インスタンス化されたスマートコントラクトのアドレスであ
る。残高照会、送金、mint, burn などの操作では、このコントラクトアドレスを指定する。

初期化 (Instantiate) チェーン上に保存されたコントラクトコードから、実際に利用可能なコントラクトイ
ンスタンスを作成する操作である。初期化メッセージにより、トークン名や初期残高などを設定する。

実行 (Execute) スマートコントラクトに対して状態変更を伴う処理を実行する操作である。CW20 では、
送金、mint, burn などが Execute に該当する。

照会 (Query) スマートコントラクトの状態を読み取る操作である。CW20 では、残高照会やトークン情報
の確認などが Query に該当する。

発行 (Mint) 新たにトークンを発行する操作である。CW20 では、mint 権限を持つアドレスのみが追加発
行を行える。

焼却 (Burn) 保有しているトークンを焼却し、供給量を減らす操作である。

許可量 (Allowance) あるアドレスに対し、自分のトークンを一定量まで利用する権限を与える仕組みであ
る。CW20 では、`IncreaseAllowance` や `DecreaseAllowance` により管理される。

初期残高 (Initial Balance) コントラクトの初期化時に特定のアドレスへ割り当てるトークン量である。
CW20 では、`initial_balances` によって設定する。

小数桁数 (Decimals) トークンの最小単位と表示単位の関係を表す数値である。たとえば `decimals` が 6 の
場合、表示上の 1 トークンは内部単位の 10^6 に相当する。

A.4 データ分析・可視化に関する用語

JSON (JavaScript Object Notation) 構造化データを保存・交換するための形式である。本実験では、ブロック情報とバリデータ情報を JSON ファイルとして保存する。

CSV (Comma-Separated Values) 表形式のデータをテキストで保存する形式である。本実験では、分析スクリプトにより生成された結果を CSV として保存する。

pandas Python のデータ分析用ライブラリである。CSV 出力や表形式データの処理に用いる。

matplotlib Python のグラフ描画ライブラリである。バリデータ数や Voting Power の推移などを可視化するために用いる。

Proposer-Visualizer ブロック提案者、ブロック間隔、バリデータ情報などを可視化するための Web アプリケーションである。取得・分析済みの CSV や JSON を読み込むことで、チェーンの挙動を視覚的に確認できる。

バリデータ遷移 (Validator Transition) ブロック高の変化に伴って、バリデータ集合、Voting Power, Proposer Priority などがどのように変化するかを表す。本実験では、Validator Transition を可視化・分析することで、合意形成に関わるノード群の変化を理解する。

時系列データ (Time-Series Data) 時間やブロック高の順序に沿って並べられたデータである。本実験では、ブロック高ごとのバリデータ数や Voting Power の変化を時系列データとして扱う。

ヒストグラム (Histogram) データの分布を棒グラフとして表す可視化手法である。本実験では、ブロック間隔の分布などを確認するために用いる。

A.5 SimBlock に関する用語

SimBlock ブロックチェーンネットワークの挙動を模擬するシミュレーションソフトウェアである。ノード数、通信遅延、ブロック生成間隔などの条件を変更し、ブロック伝播や分岐の発生を観察できる。

ブロック伝播 (Block Propagation) あるノードで生成されたブロックが、他のノードへ送信され、ネットワーク全体に共有されていく過程である。

ブロック分岐 (Fork) 複数のノードがほぼ同時にブロックを生成した結果、一時的に複数の候補チェーンが存在する状態である。

孤立ブロック (Orphan Block) 分岐の結果、最終的に正規チェーンに採用されなかったブロックである。

ネットワーク遅延 (Network Latency) あるノードから別のノードへデータが届くまでにかかる時間である。ネットワーク遅延が大きいほど、ブロック分岐が発生しやすくなる。

ブロック生成間隔 (Block Generation Interval) 連続するブロックが生成される時間間隔である。ブロック生成間隔が短く、ブロック伝播時間が長い場合、分岐が起こりやすくなる。

帯域幅 (Bandwidth) ネットワーク上で単位時間あたりに送受信できるデータ量である。帯域幅が小さい場合、ブロック伝播に時間がかかり、分岐発生に影響する可能性がある。

ノード接続数 (Node Degree) あるノードが接続している他ノードの数である。接続数は、ブロックやトランザクションの伝播速度に影響する。